

# Maximum Defective Biclique Search in Large Bipartite Graphs

## ABSTRACT

The problem of identifying the maximum edge biclique in bipartite graphs has attracted considerable attention in bipartite graph analysis, with numerous real-world applications such as fraud detection, community detection, and online recommendation systems. However, real-world graphs frequently contain noise or incomplete information, leading to overly restrictive conditions when employing the biclique model. To mitigate this, we propose a relaxed subgraph model, called the  $k$ -defective biclique, which allows for up to  $k$  missing edges compared to the traditional biclique model. We then investigate the problem of finding the maximum edge  $k$ -defective biclique in a bipartite graph. We prove that the problem is NP-hard. To tackle this issue, we propose a novel algorithm that leverages a new branch-and-bound framework with a worst-case time complexity of  $O(m\alpha_k^n)$ , where  $\alpha_k < 2$ . We further enhance this approach by incorporating a novel pivoting technique, reducing the worst-case time complexity to  $O(m\beta_k^n)$ , where  $\beta_k$  is a real number strictly smaller than  $\alpha_k$ . In addition, we develop a series of optimization techniques, including graph reduction, novel upper bounds, and a heuristic algorithm to further improve practical performance. Extensive experiments on 10 large real-world datasets validate the efficiency and effectiveness of our proposed methods. The results indicate that our algorithms consistently outperform the state-of-the-art, offering up to  $1000\times$  speedups across various parameter settings.

## 1 INTRODUCTION

Bipartite graph is a commonly encountered graph structure consisting of two disjoint vertex sets and an edge set, where each edge connects vertices from the two different sets. In recent decades, bipartite graphs have been utilized as a fundamental tool for modeling various binary relationships, such as user-product interactions in e-commerce [28], authorship connections between authors and publications [27], and associations between genes and phenotypes in bioinformatics [15, 36]. Mining cohesive subgraphs from bipartite graphs has proven to be critically important across diverse domains [13, 42]. For instance, in reviewer-product graphs, where edges represent review relationships, identifying cohesive subgraphs facilitates the detection of unusual patterns or anomalies in product reviews, aiding in the identification of fraudulent reviews or camouflage attacks [22].

The biclique, which connects all vertices on both sides, serves as a fundamental cohesive subgraph structure [29, 34, 43] due to

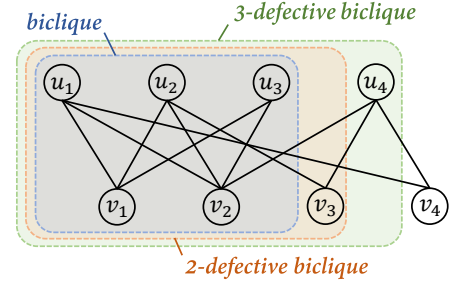


Figure 1: An example graph with two defective bicliques

its wide applications in tasks such as fraud detection [4], community detection [1, 26] and bioinformatics network analysis [25]. Extensive methods for maximum biclique search have been developed in recent years [1, 13, 16, 20, 31]. However, real-world graphs often contain noisy or incomplete information [30], making the traditional biclique model overly restrictive for capturing cohesive subgraphs on such a kind of bipartite graphs. More relaxed biclique models offer a promising alternative to address these challenges.

Inspired by the widely studied  $k$ -defective clique model in traditional graphs [9, 10, 45], we propose a new cohesive subgraph model for bipartite graphs, termed the  $k$ -defective biclique, which serves as a relaxation of the traditional biclique. Formally, a subgraph in the bipartite graphs is defined as a  $k$ -defective biclique if it has at most  $k$  missing edges. For instance, Fig. 1 illustrates a biclique graph  $G$  and two defective bicliques, where the 2-defective biclique has two missing edge  $(u_1, v_3)$  and  $(u_3, v_3)$ , and the 3-defective biclique has three missing edges,  $(u_1, v_3)$ ,  $(u_3, v_3)$ , and  $(u_4, v_1)$ . Notably, the  $k$ -defective biclique reduces to a standard biclique when  $k = 0$ , highlighting its generalization of the biclique structure.

It can be seen that the  $k$ -defective biclique offers greater flexibility than the traditional bicliques, since the density can be controlled by adjusting  $k$  to suit specific requirements. In this paper, we focus mainly on the problem of identifying a maximum edge  $k$ -defective biclique, i.e., a  $k$ -defective biclique with the most number of edges in a bipartite graph. For simplicity, we refer to this as the maximum  $k$ -defective biclique unless otherwise specified. We prove that the maximum  $k$ -defective biclique search problem is NP-hard, indicating that no polynomial-time algorithms exist for solving this problem unless NP=P. Notably, existing  $k$ -defective clique algorithms [7, 9, 10, 18] on traditional graphs are not easily applicable to our problem due to several challenges: (1) converting a bipartite graph into a traditional graph by pairwise connecting the vertices on each side leads to significant memory overhead; (2) the maximum edge  $k$ -defective biclique in the bipartite graph does not correspond to the maximum  $k$ -defective clique in the converted graph due to additional edges introduced during conversion; and (3) existing algorithms are often inefficient when applied to the dense graphs resulting from such conversions.

**Contributions.** To efficiently address the maximum  $k$ -defective biclique search problem, we conduct an in-depth study of its properties and develop two novel algorithms with the worst-case time complexity less than  $O^*(2^n)$ . Additionally, we also present a series

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).  
Conference SIGMOD'25, June 22–27, 2025, Berlin, Germany

of optimization techniques to further improve the efficiency of our algorithms. Finally, we evaluate the effectiveness and efficiency of our methods through comprehensive experiments. The main contributions of this paper are summarized as follows:

*A new cohesive subgraph model.* We propose a new model of cohesive subgraphs for bipartite graphs, termed the  $k$ -defective biclique, defined as a subgraph with at most  $k$  missing edges. It can be served as an alternative to the traditional biclique for representing cohesive subgraphs in bipartite graphs, particularly in the presence of noise or lower density requirements. We then demonstrate that the problem of finding the maximum  $k$ -defective biclique in bipartite graphs is NP-hard.

*Two Novel Search Algorithms.* To address the problem of finding the maximum  $k$ -defective biclique, we propose two novel algorithms with nontrivial worst-case time complexity guarantees. The first algorithm utilizes a branch-and-bound technique combined with a newly-developed binary branching strategy. We show that the time complexity of this algorithm is bounded by  $O(m\alpha_k^n)$ , where  $n$  and  $m$  represent the total number of vertices and edges of the bipartite graph, respectively, and  $\alpha_k$  is a positive real number strictly less than 2 (e.g., for  $k = 1, 2, 3$ ,  $\alpha_k = 1.911, 1.979$ , and  $1.995$ , respectively). The second algorithm employs an innovative pivoting-based branching strategy that effectively integrates with our binary branching strategy. We prove that this pivot-based (or the second) algorithm achieves a lower time complexity of  $O(m\beta_k^n)$ , where  $\beta_k$  is a positive real number strictly less than  $\alpha_k$ . (e.g., when  $k = 1, 2, 3$ ,  $\beta_k = 1.717, 1.856$ , and  $1.931$ , respectively).

*Nontrivial optimization techniques.* We also develop a series of non-trivial optimization techniques to improve the efficiency of our proposed algorithms, including graph reduction techniques, novel upper-bound techniques, and a heuristic algorithm. Specifically, the graph reduction techniques focus on reducing the size of the input bipartite graph based on constraints such as degree, core number [5], and the number of common neighbors. In the branch-and-bound process, we devise tight vertex and edge upper bounds by carefully considering the constraints on the number of vertices, edges, and missing edges in the  $k$ -defective biclique, to minimize unnecessary branching. We then design an efficient algorithm to compute the proposed upper bounds with linear time complexity. Additionally, the heuristic algorithm incrementally expands subsets in a specific order to obtain an initial large  $k$ -defective biclique, which helps filter out smaller  $k$ -defective bicliques from further computation.

*Comprehensive experimental studies.* We conduct extensive experiments to evaluate the efficiency, effectiveness, and scalability of our algorithms on 10 large real-world graphs. The experimental results show that our algorithms consistently outperform the state-of-the-art baseline method [12] across all parameter settings. In particular, our algorithm achieves improvements of up to three orders of magnitude on most datasets. For instance, on the *Twitter* (9M vertices, 10M edges) dataset with  $k = 2$ , our algorithm completes in 10 seconds, whereas the baseline method cannot finish the calculations within 3 hours. Additionally, we perform a case study on a fraud detection task to show the high effectiveness of our solutions in real-world applications. The source code of our

algorithms is available anonymously at <https://anonymous.4open.science/r/MaximumDefectiveBiclique-0B4C>.

## 2 PRELIMINARIES

Let  $G = (U, V, E)$  be an undirected bipartite graph, where  $U$  and  $V$  are two disjoint sets of vertices, and  $E \subseteq U \times V$  is the set of edges. Denote by  $n = |U| + |V|$  and  $m = |E|$  the total number of vertices and edges in  $G$ , respectively. Denote by  $S = (U_S, V_S)$  a vertex subset of  $G$ , where  $U_S \subseteq U$  and  $V_S \subseteq V$ . Let  $v \in S$  indicate that  $v \in U_S \cup V_S$ . Let  $G(S) = (U_S, V_S, E_S)$  be the subgraph of  $G$  induced by  $S$ , where  $E_S = \{(u, v) \mid u \in U_S \wedge v \in V_S \wedge (u, v) \in E\}$ . For  $u \in U$ , denote by  $N(u) = \{v \in V, \mid (u, v) \in E\}$  and  $\bar{N}(u) = \{v \in V \mid (u, v) \notin E\}$  the set of neighbors and non-neighbors of  $u$  in  $G$ , respectively. Similar definitions apply for the vertices in  $U$ . Let  $d(u) = |N(u)|$  and  $\bar{d}(u) = |\bar{N}(u)|$  be the degree and non-degree of  $u$  in  $G$ , respectively. For a vertex subset (or a subgraph)  $S$  of  $G$ , let  $N_S(u) = N(u) \cap (U_S \cup V_S)$  and  $\bar{N}_S(u) = \{v \in S \setminus N(u) \mid u \text{ and } v \text{ are on opposite sides}\}$  be the set of  $u$ 's neighbors and non-neighbors in  $S$ , respectively. Let  $d_S(u) = |N_S(u)|$  and  $\bar{d}_S(u) = |\bar{N}_S(u)|$  be the degree and non-degree of  $u$  in  $S$ , respectively. For a bipartite graph  $G = (U, V, E)$ , we refer to  $(u, v) \in U \times V$  as a non-edge of  $G$  if  $(u, v) \notin E$ . We denote  $\bar{E}_S$  the set of non-edges in  $G(S)$ , i.e.,  $\bar{E}_S = U_S \times V_S \setminus E_S$ . The formal definition of  $k$ -defective biclique is given below.

*Definition 1 ( $k$ -defective biclique).* Given a bipartite graph  $G = (U, V, E)$ , a  $k$ -defective biclique  $D = (U_D, V_D, E_D)$  of  $G$  is a subgraph of  $G$  that has at most  $k$  non-edges, i.e.,  $|U_D \times V_D \setminus E_D| \leq k$ .

As can be seen, the  $k$ -defective biclique corresponds to the biclique if  $k = 0$ , which indicates that  $k$ -defective biclique represents a more general structure. This flexibility allows for its application in various domains where bicliques are utilized.

Given a  $k$ -defective biclique  $D$  of  $G$ , if it contains the most number of edges among all  $k$ -defective biclique of  $G$ , we refer to it as a maximum  $k$ -defective biclique in  $G$ . To facilitate the illustration, we denote the maximum  $k$ -defective biclique as  $k$ -MDB.

Inspired by the maximum biclique search problem, in this paper, we focus on the problem of searching for a  $k$ -MDB in a bipartite graph. However, we observe that a  $k$ -MDBs in many real-world graphs may exhibit a skewed structure, similar to that of maximum bicliques, where one side has a large number of vertices while the other side may contain only a few (or even one) vertices. Such  $k$ -MDBs often hold limited practical values in network analysis. To address this issue, we define a size threshold  $\theta$ , representing the minimum number of vertices required on both sides of the  $k$ -MDB. Furthermore, we impose the condition  $\theta > k$  to ensure that the  $k$ -MDB is connected, as demonstrated in the following lemma.

**LEMMA 1.** Given a bipartite graph  $G = (U, V, E)$  and a  $k$ -defective biclique  $D = (U_D, V_D, E_D)$  of  $G$ , if  $|U_D| > k$  and  $|V_D| > k$ , we establish that  $D$  is a connected subgraph in  $G$ .

**PROOF.** Assume that a  $k$ -defective biclique  $D$  is not connected when  $|U_D| > k$  and  $|V_D| > k$ . In this case,  $D$  can be decomposed into two nonempty subgraphs  $D_1 = (U_{D_1}, V_{D_1}, E_{D_1})$  and  $D_2 = (U_{D_2}, V_{D_2}, E_{D_2})$ , with the results of  $E_{D_1} \cup E_{D_2} \subseteq E_D$  and  $|E_{D_1}| + |E_{D_2}| = |E_D|$ . The number of non-edges between  $D_1$  and  $D_2$  is given by  $|U_{D_1}| \cdot |V_{D_2}| + |U_{D_2}| \cdot |V_{D_1}|$ , which exceeds  $|U_D|$  (or  $|V_D|$ ) and

is thus larger than  $k$ . This contradicts the assumption that  $D$  is a  $k$ -defective biclique. Therefore, the proof is complete.  $\square$

Based on Lemma 1, we formally define the problem addressed in this paper as follows.

**Problem statement** (MDB search problem). Given a bipartite graph  $G = (U, V, E)$  and two integers  $k$  and  $\theta$  with  $\theta > k$ , the MDB search problem aims to find a connected  $k$ -MDB  $D = (U_D, V_D, E_D)$  of  $G$  satisfying  $|U_D| \geq \theta$  and  $|V_D| \geq \theta$ .

**THEOREM 2.** The MDB search problem is NP-hard.

**PROOF.** We prove this by reducing the maximum biclique search problem to the 1-MDB search problem. We call a biclique in a bipartite graph  $G = (U, V, E)$  is the maximum biclique if it contains the most number of edges among all biclique of  $G$ . The maximum biclique search problem in  $G$  is known to be NP-hard [35]. Consider a new constructed graph  $G' = (U', V', E')$ , where  $U' = U \cup \{u'\}$  and  $V' = V \cup \{v'\}$ , and  $E' = E \cup \{(u', v) \mid v \in V\} \cup \{(u, v') \mid u \in U\}$ . Notably, the 1-MDB of  $G'$  consistently includes  $u'$  and  $v'$ , as any 1-MDB in which the non-edge is replaced by  $(u', v')$  is still a 1-MDB. Thus, it follows that a subgraph  $S$  of  $G$  is a maximum biclique if and only if  $S \cup \{u', v'\}$  is a 1-MDB of  $G'$ . Therefore, we conclude that the  $k$ -MDB search problem is NP-hard.  $\square$

### 3 OUR PROPOSED ALGORITHMS

In this section, we propose two novel algorithms to solve the  $k$ -MDB search problem. Specifically, we first propose a new branch-and-bound algorithm that utilizes a newly-designed binary branching strategy, achieving the worst-case time complexity of  $O(m\alpha_k^n)$ , where  $\alpha_k < 2$  (e.g.  $k = 1, 2$ , and  $3$ , we have  $\alpha_k = 1.911, 1.979$ , and  $1.995$ , respectively). To further improve the efficiency, we propose a novel pivoting-based algorithm that incorporates a well-designed pivoting-based branching strategy with our proposed binary branching strategy. We prove that the time complexity of the improved algorithm is bounded by  $O(m\beta_k^n)$ , where  $\beta_k < \alpha_k$  (e.g., when  $k = 1, 2, 3$ ,  $\beta_k = 1.717, 1.856$ , and  $1.931$ , respectively). To the best of our knowledge, the algorithms we proposed in this section are the first two  $k$ -MDB search algorithms that surpass the trivial time complexity of  $O^*(2^n)$ .

#### 3.1 A New Branch-and-bound Algorithm

We firstly introduce a branch-and-bound algorithm for the  $k$ -MDB problem. It employs a straightforward branch-and-bound approach by selecting a vertex  $v$  from the graph and dividing the  $k$ -MDB search problem into two subproblems: one seeks a  $k$ -MDB that includes  $v$ , while the other seeks a  $k$ -MDB that excludes  $v$ . Specifically, given a bipartite graph  $G$ , denote by  $S = (U_S, V_S)$  a partial set and  $C = (U_C, V_C)$  a candidate set, where  $G(S)$  is a  $k$ -defective biclique of  $G$ , and for any vertex  $u \in C$ ,  $G(S \cup \{u\})$  forms a larger  $k$ -defective biclique of  $G$ .  $(S, C)$  is referred to as an instance, which aims to identify a  $k$ -MDB consisting of  $S$  and a subset of  $C$ . In particular, the instance  $((\emptyset, \emptyset), (U, V))$  aims to find a  $k$ -MDB of  $G$ , while the instance  $(S^*, (\emptyset, \emptyset))$  corresponds to a possible solution  $G(S^*)$ . To clarify, we refer to a vertex  $u \in C$  as a *branching vertex* when a new sub-instance is generated by expanding  $S$  with  $u$ .

Given a specific instance  $(S, C)$ , our branch-and-bound algorithm selects a branching vertex  $u \in C$  and generates two new sub-instances:  $(S \cup \{u\}, C \setminus \{u\})$  and  $(S, C \setminus \{u\})$ . The algorithm then recursively processes each sub-instance in the same manner until the candidate set becomes empty, where the partial set yields a possible solution. Finally, the  $k$ -MDB of  $G$  will be obtained from the one with most edges among all yielded solutions. The simplest approach to obtain a branching vertex is to randomly select one from  $C$ . However, this strategy may lead to the selection of all vertices in  $C$  leading to a worst-case time complexity of  $O^*(2^n)$  and too many unnecessary computations. To improve the efficiency and reduce the time complexity, we propose a new binary branching strategy. Below, we first present two facts to exclude some unnecessary vertices from  $C$ .

**FACT 1.** Given a bipartite graph  $G$  and an instance  $(S, C)$ , for each vertex  $v \in C$  that satisfies  $\bar{d}_{S \cup C}(v) = 0$ , moving  $v$  from  $C$  to  $S$  does not change the result.

**FACT 2.** Given a bipartite graph  $G$  and an instance  $(S, C)$ , for each vertex  $v \in C$  that satisfies  $\bar{d}_S(v) > k - \bar{d}(S)$ , removing  $v$  from  $C$  does not change the result.

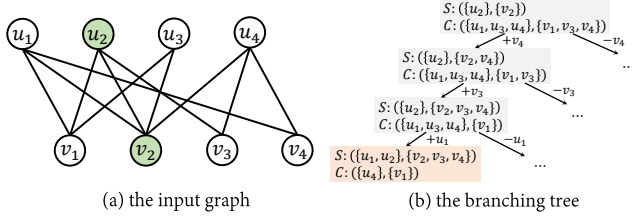
The correctness of Fact 1 and 2 follows directly from the definition of  $k$ -defective biclique. Fact 1 ensures that every vertex in  $C$  has at least one non-neighbor in  $G(S \cup C)$ , while fact 2 guarantees that for any vertex  $v$  in  $C$ ,  $G(S \cup \{v\})$  forms a valid  $k$ -defective biclique. We assume that the instance  $(S, C)$  is established based on these two facts. Next, we consider the problem of selecting the branching vertex.

Intuitively, if  $k$  vertices from  $C$  with non-neighbors in  $S$  are successively added to  $S$ , then  $S$  will contain at least  $k$  non-edges. Consequently, this implies that all vertices in  $C$  must be the neighbors of each  $v \in S$  with  $\bar{d}_S(v) > 0$ . Additionally, if no vertex in  $C$  that has non-neighbors in  $S$ , adding a vertex  $v$  from  $C$  to  $S$  will result in exactly  $\bar{d}_C(v)$  vertices in  $C$  with non-neighbors in  $S$ . Based on these observations, we give our new binary branching strategy as follows.

**New binary branching strategy.** Given an instance  $(S, C)$ , let  $u = \arg \max_{u \in C} \bar{d}_S(u)$ . The sub-instances are generated in the following way:

- If  $\bar{d}_S(u) > 0$ , select  $u$  as the branching vertex and generate two sub-instances:  $(S \cup \{u\}, C \setminus \{u\})$ ,  $(S, C \setminus \{u\})$ .
- Otherwise, select  $v \in C$  with the maximum  $\bar{d}_C(v)$  as the branching vertex, and generate two sub-instances:  $(S \cup \{v\}, C \setminus \{v\})$ ,  $(S, C \setminus \{v\})$ .

**Example 3.** Fig. 2 shows an example of our new binary branching strategy for finding a 2-MDB. The input graph is depicted in Fig. 2(a), where the current instance is  $S = (\{u_2\}, \{v_2\})$  (green vertices) and  $C = (\{u_1, u_3, u_4\}, \{v_1, v_3, v_4\})$  (uncolored vertices). Fig. 2(b) illustrates the branching tree obtained by applying our binary branching strategy. Specifically, in the top-level branch,  $v_4$  is selected as the branching vertex due to its maximum number of non-neighbors in  $S$ . When  $v_4$  is added to  $S$ , it brings a non-edge in  $S \cup \{v_4\}$ . Then, we observe that there is no vertex in the second-level branch with non-neighbors in  $S \cup \{v_4\}$ . Therefore,  $v_3$  is selected as the second-level branching vertex as it has the most non-neighbors



**Figure 2: An example of the new binary branching strategy, where  $k = 2$  and current instance is set as  $S = (\{u_2\}, \{v_2\})$  (green vertices) and  $C = (\{u_1, u_3, u_4\}, \{v_1, v_3, v_4\})$  (uncolored vertices).**

**Algorithm 1: The branch-and-bound algorithm**

---

**Input:** Bipartite graph  $G = (U, V, E)$ , integers  $k$  and  $\theta$   
**Output:** A  $k$ -MDB of  $G$

```

1  $D^* \leftarrow$  an empty graph;
2 BranchB( $(\emptyset, \emptyset), (U, V)$ );
3 return  $D^*$ ;
4 Procedure BranchB( $S = (U_S, V_S), C = (U_C, V_C)$ ):
5   if  $C = \emptyset$  then
6     if  $|E_G(S)| > |E_{D^*}|$  and  $|U_S| \geq \theta$  and  $|V_S| \geq \theta$  then
7        $D^* \leftarrow G(S)$ ;
8     return;
9    $u \leftarrow \arg \max_{v \in C} \bar{d}_S(v)$ ;
10  if  $\bar{d}_S(u) = 0$  then  $u \leftarrow \arg \max_{v \in C} \bar{d}_C(v)$ ;
11   $(S', C') \leftarrow \text{Update}(S, C, u)$ ;
12  BranchB( $S' \cup \{u\}, C'$ ); BranchB( $S, C \setminus \{u\}$ );
13 Function Update( $S, C, u$ ):
14   $C' \leftarrow \{v \in C \setminus \{u\} \mid \bar{d}_{S \cup \{u\}}(v) \leq k - \bar{d}(S)\}$ ;
15   $C'_0 \leftarrow \{v \in C' \mid \bar{d}_{S \cup \{u\} \cup C'}(v) = 0\}$ ;
16  return  $(S \cup C'_0, C' \setminus C'_0)$ ;
```

---

in  $C \setminus \{v_4\}$ . In the third-level branch,  $u_1$  is selected as the branching vertex as it has the most non-neighbors in  $S \cup \{v_4, v_3\}$ . In the forth-level,  $S \cup \{u_1, v_3, v_4\}$  contains 2 non-edges, leading to the deletion of  $u_3$  from  $C$  since it still has non-neighbors in  $S \cup \{u_1, v_3, v_4\}$ , which is the key to reducing the unnecessary computations.

**Implementation details.** Algorithm 1 outlines the pseudocode of our proposed branch-and-bound algorithm. It takes a bipartite graph  $G$  and two integers  $k$  and  $\theta$  as input parameters, and returns a  $k$ -MDB  $D^*$  of  $G$ . It first initializes  $D^*$  as an empty graph, and then starts to search  $k$ -MDB by invoking the procedure BranchB where all vertices in  $G$  serve as the candidate set (line 2). BranchB holds an instance  $(S, C)$  as its parameter, where  $S$  denotes the partial set and  $C$  denotes the candidate set. Within BranchB, the algorithm first determines whether a larger  $k$ -defective biclique has been found (lines 5-8). After that, it selects a branching vertex  $u$  from  $C$  using our new binary branching strategy (lines 9-10) and generates two sub-instance  $(S', C' \cup \{u\})$  and  $(S, C \setminus \{u\})$  (lines 11-12). Here,  $S'$  and  $C'$  are the new partial set and new candidate set derived by invoking Update (line 11), which applies the Fact 1 and Fact 2 on  $(S, C)$  (lines 14-15).

We next analyze the time complexity of Algorithm 1, which is presented in the following theorem.

**THEOREM 4.** Given a bipartite graph  $G = (U, V, E)$  and an integer  $k$ , the time complexity Algorithm 1 is given by  $O(m\alpha_k^n)$ , where  $\alpha_k$  is the largest real root of equation  $x^{2k+5} - 2x^{2k+4} + x^3 - x^2 + 1 = 0$ . Specifically, when  $k = 1, 2$  and  $3$ , we have  $\alpha_k = 1.911, 1.979$  and  $1.995$ , respectively.

**PROOF.** Let  $T(n)$  denote the maximum number of leave instances generated by BranchB (i.e. instances with an empty  $C$ ), where  $n = |C| = |U_C| + |V_C|$ . Since each recursive call of BranchB takes  $O(m)$  time, the total time complexity of BranchB is  $O(m \cdot T(n))$ .

Then, we analyze the size of  $T(n)$  based on the binary branching strategy. In each branch of BranchB, either a vertex with at least one non-neighbor in  $S$  or a vertex with the most non-neighbors in  $C$  is selected as the branching vertex. If the vertex  $u$  with at least one non-neighbor in  $S$  is selected, it results in at least one additional non-edge in  $S \cup \{u\}$ . Alternatively, if a vertex  $v$  with the most non-neighbors in  $C$  is selected as the branching vertex, the next-level branch will contain at least  $\bar{d}_C(v)$  vertices in  $C$  that have non-neighbors in  $S \cup \{v\}$ , where  $\bar{d}_C(v) > 0$  according to Fact 1. Consequently, we can conclude that in the worst case, adding two vertices from  $C$  to  $S$  will result in at least one additional non-edge in  $S$ . After adding at most  $2k+1$  vertices from  $C$  to  $S$ , we have  $|E_S| = k$  and there exists a vertex  $v \in C$  such that  $\bar{d}_S(v) > 0$ . Based on this, the recurrence is given by  $T(n) \leq \sum_{i=1}^{2k+1} T(n-i) + T(n-2k-2)$ .

Note that the  $(2k+2)$ -th level of branching (the term  $T(n-2k-2)$ ) corresponds to a branch that searches for the maximum biclique. In this case, it is evident that any vertex in  $C$  selected as the branching vertex will result in at least one additional vertex being removed from  $C$  according to 1. Thus, we yield a recurrence of  $T(n-2k-2) = T(n-2k-3) + T(n-2k-4)$  for such a branch. Based on above discussion, we can present the following tighter recurrence.

$$T(n) \leq \sum_{i=1}^{2k+1} T(n-i) + T(n-2k-3) + T(n-2k-4). \quad (1)$$

According to the theory proposed in [17],  $T(n)$  can be expressed in the form of  $\alpha_k^n$ , where the value of  $\alpha_k$  derives from the largest real root of equation  $x^n = x^{n-1} + x^{n-2} + \dots + x^{n-2k-1} + x^{n-2k-3} + x^{n-2k-4}$ , which can be simplified to  $x^{2k+5} - 2x^{2k+4} + x^3 - x^2 + 1 = 0$ . Therefore, the theorem is proven.  $\square$

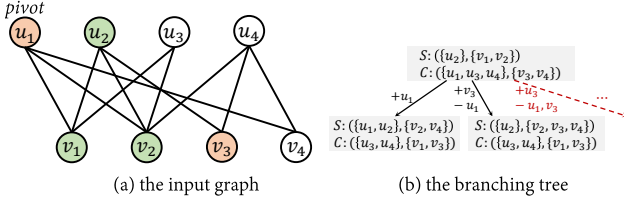
### 3.2 A Novel Pivoting-based Algorithm

We observe that Algorithm 1 may still generate numerous unnecessary sub-instances. Consider an instance  $(S, C)$  and a branching vertex  $v \in C$  with  $\bar{d}_S(v) = 0$ . In Algorithm 1, if  $v$  and all non-neighbors of  $v$  have been excluded from the current instance, the remaining vertices cannot form any  $k$ -MDB. This is because any  $k$ -defective biclique derived from these remaining vertices can always be expanded by including  $v$ . Consecutively, Algorithm 1 may produce many  $k$ -defective bicliques that are not maximum. To address the problem and further improve the efficiency, we propose a novel pivoting-based algorithm in this section. We formalize above insights into the following lemma.

**LEMMA 2.** Given an instance  $(S, C)$  and any vertex  $v \in C$  with  $\bar{d}_S(v) = 0$ , the  $k$ -MDB of instance  $(S, C)$  must contain either  $v$  or a vertex in  $\bar{N}_C(v)$ .

Based on Lemma 2, we develop a pivoting strategy that provides a new way to generate sub-instances. The details are presented as follows.

**Pivoting strategy.** Consider an instance  $(S, C)$  and a vertex  $v \in C$  with  $\bar{d}_S(v) = 0$ . Let  $u$  be the *pivot vertex*. Assume  $\bar{N}_C(u) =$



**Figure 3: An example of the pivoting-based branching strategy, where  $k = 2$  and current instance is set as  $S = (\{u_2\}, \{v_2\})$  (green vertices) and  $C = (\{u_1, u_3, u_4\}, \{v_1, v_3, v_4\})$  (orange and uncolored vertices).**

$\{v_1, v_2, \dots, v_r\}$ , where  $r = \bar{d}_C(v)$ . The pivoting strategy treats each vertex in  $\{u\} \cup \bar{N}_C(u)$  as a branching vertex and generates exactly  $r + 1$  sub-instances. The first sub-instance is given by  $(S \cup \{u\}, C \setminus \{u\})$ , and for  $i \geq 2$ , the  $i$ -th sub-instance is given by  $(S \cup \{v_{i-1}\}, C \setminus \{u, v_1, v_2, \dots, v_{i-1}\})$ .

With this strategy,  $S$  is expanded only with the vertex  $u$  and vertices in  $\bar{N}_C(u)$ , avoiding redundant sub-instances that would lead to non-maximum  $k$ -defective biclique. Next, we consider the selection of the pivot vertex. Let  $C_0 \subseteq C$  be the set of vertices fully connected to  $S$ , i.e.,  $C_0 = \{v \in C \mid \bar{d}_S(v) = 0\}$ . According to the pivoting strategy, any vertex from  $C_0$  can serve as a pivot vertex. To minimize the number of sub-instances, the vertex  $u \in C_0$  with the fewest non-neighbors in  $C$  is chosen as the pivot vertex.

In the case where  $C_0$  is empty, every vertex in  $C$  has at least one non-neighbor in  $S$ , making the pivoting strategy inapplicable. Fortunately, as discussed in Sec. 3.1, continuously branching  $k$  times by adding vertices from  $C$  to  $S$  will result in  $S$  containing at least  $k$  non-edges. Moreover,  $C$  becomes an empty set after this process, which significantly reduces unnecessary computations. Motivated by this observation, we integrate the pivoting strategy with the binary branching strategy to develop a novel pivoting-based branching strategy as outlined below.

**Novel pivoting-based branching strategy.** Given an instance  $(S, C)$ , let  $C_0 = \{v \in C \mid \bar{d}_S(v) = 0\}$ . The new sub-instances of  $(S, C)$  are generated in the following way:

- If  $C_0$  is empty or  $|C \setminus C_0| > k - |\bar{E}_S|$ , let  $u = \arg \max_{v \in C_0} \bar{d}_S(v)$  be the branching vertex and generate two sub-instances:  $(S \cup \{u\}, C \setminus \{u\})$  and  $(S, C \setminus \{u\})$ .
- Otherwise, let  $u = \arg \min_{v \in C_0} \bar{d}_C(v)$ , and then:
  - If  $\bar{d}_C(u) > k - |\bar{E}_S| > 0$ , generate two sub-instances with  $u$  as the branching vertex:  $(S \cup \{u\}, C \setminus \{u\})$  and  $(S, C \setminus \{u\})$ .
  - Otherwise, generate  $\bar{d}_C(u) + 1$  sub-instances with  $u$  as the pivot vertex, where the first sub-instance is given by  $(S \cup \{u\}, C \setminus \{u\})$ , and for  $i \geq 2$ , the  $i$ -th sub-instance is given by  $(S \cup \{v_{i-1}\}, C \setminus \{u, v_1, v_2, \dots, v_{i-1}\})$ .

**Example 5.** Fig. 3 shows an example of the our pivoting-based branching strategy for the 2-MDB search. The example graph is given by Fig. 3(a), with the current instance set as  $S = (\{u_2\}, \{v_1, v_2\})$  (green vertices) and  $C = (\{u_1, u_3, u_4\}, \{v_3, v_4\})$  (orange and uncolored vertices). According to the pivoting-based branching strategy,  $C_0 = (\{u_1, u_3\}, \{v_3\})$ , and  $u_1$  is selected as the pivot vertex since it has the fewest non-neighbors in  $C$  among vertices in  $C_0$ . Then, only  $u_1$  and all non-neighbors of  $u_1$  in  $C$  are used to generate

## Algorithm 2: The pivoting-based algorithm

---

**Input:** Bipartite graph  $G = (U, V, E)$ , integers  $k$  and  $\theta$   
**Output:** A  $k$ -MDB of  $G$

```

1  $D^* \leftarrow$  an empty graph;
2 BranchP( $(\emptyset, \emptyset), (U, V)$ );
3 return  $D^*$ ;
4 Procedure BranchP( $S = (U_S, V_S), C = (U_C, V_C)$ ):
5   if  $C = \emptyset$  then
6     if  $|E_{G(S)}| > |E_{D^*}|$  and  $|U_S| \geq \theta$  and  $|V_S| \geq \theta$  then
7        $D^* \leftarrow G(S)$ ;
8     return;
9    $C_0 \leftarrow \{v \in C \mid \bar{d}_S(v) = 0\}$ ;
10  if  $C_0 = \emptyset$  or  $|C \setminus C_0| > k - |\bar{E}_S|$  then
11     $u \leftarrow \arg \max_{v \in C_0} \bar{d}_S(v)$ ;
12     $(S', C') \leftarrow \text{Update}(S, C, u)$ ;
13    BranchP( $S' \cup \{u\}, C'$ ); BranchP( $S, C \setminus \{u\}$ );
14  else
15     $u \leftarrow \arg \min_{v \in C_0} \bar{d}_C(v)$ ;
16    if  $\bar{d}_C(u) > k - |\bar{E}_S| > 0$  then
17       $(S', C') \leftarrow \text{Update}(S, C, u)$ ;
18      BranchP( $S' \cup \{u\}, C'$ ); BranchP( $S, C \setminus \{u\}$ );
19    else
20      for  $v \in \{u\} \cup \bar{N}_C(u)$  do
21         $(S', C') \leftarrow \text{Update}(S, C, v)$ ;
22        BranchP( $S' \cup \{v\}, C'$ );  $C \leftarrow C \setminus \{v\}$ ;

```

---

new sub-instances (orange vertices), while other vertices of  $C$  are not considered (uncolored vertices). This results in only two sub-instances generated by our pivot-based branching strategy. The branching tree for this example is illustrated in Fig. 3(b), where the black arrows represent the generated branches while the red arrows indicate the unnecessary branches.

**Implementation details.** Algorithm 2 outlines the pseudocode of the proposed pivot-based algorithm. It begins by searching for  $k$ -MDB by invoking the procedure BranchP with an empty partial set  $S$  and the entire vertex set of  $G$  as the candidate set  $C$  (line 2). The algorithm returns the global variable  $D^*$  as the final  $k$ -MDB after BranchP is processed (line 3). Within BranchP, the algorithm first updates  $D^*$  if a larger  $k$ -defective biclique is found (lines 5-7). Then, it generates new sub-instances based on the pivoting-based branching strategy (lines 8-22). Specifically, if  $C_0 = \emptyset$  or  $|C \setminus C_0| > k - |\bar{E}_S|$ , the vertex  $u = \arg \max_{v \in C_0} \bar{d}_S(v)$  is selected as the branching vertex (line 11), and two new sub-instances are generated (line 13). Otherwise, the algorithm assigns  $u = \arg \min_{v \in C_0} \bar{d}_C(v)$ . If  $\bar{d}_C(u) > k - |\bar{E}_S|$ , two sub-instances are generated with  $u$  as the branching vertex (lines 16-18). Otherwise,  $\bar{d}_C(u) + 1$  new sub-instances are generated with  $u$  and all  $u$ 's non-neighbors in  $C$  as branching vertex (lines 20-22). When a branching vertex is added to  $S$ , the algorithm adopts the Update procedure presented in Algorithm 1 to update the current instance based on Fact 1 and Fact 2 (lines 12, 17 and 21).

Below, we analyze the time complexity of Algorithm 2.

**THEOREM 6.** Given a bipartite graph  $G = (U, V, E)$  and an integer  $k$ , the time complexity of Algorithm 2 is given by  $O(m\beta_k^n)$ , where  $\beta_k$  is the largest real root of equation  $x^{2k+5} - 2x^{2k+4} + x^{k+3} - 2x + 2 = 0$ . Specifically, when  $k = 1, 2$  and  $3$ , we have  $\beta_k = 1.717, 1.856$  and  $1.931$ , respectively.

PROOF. In Algorithm 2, Let  $T(n)$  denote the maximum number of leave instances generated by BranchP, where  $n = |C| = |U_C| + |V_C|$ . It is easy to verify that finding the vertex with most (least) non-neighbors in  $S(C)$  takes at most  $O(m)$  time. Then, each recursive call of BranchP takes  $O(m)$  time, and thus BranchP takes  $O(m \cdot T(n))$  time. We next focus on analyzing the size of  $T(n)$ . Based on the pivoting-based branching strategy, there are three cases that need to be discussed.

- (1) If  $C_0 = \emptyset$  or  $|C \setminus C_0| > k - |\bar{E}(S)|$ , BranchP continuously applies the binary branching strategy. After moving at most  $k$  vertices from  $C$  to  $S$ ,  $S$  contains  $k$  non-edges. At this point, at least one vertex in  $C$  will be removed as it has non-neighbors in  $S$ . Therefore, the recurrence for this case is given by  $T(n) \leq \sum_{i=1}^{k+1} T(n-i)$ .
- (2) If  $k - |\bar{E}_S| = 0$ , the term  $T(n)$  corresponds to a maximum biclique search problem, and BranchP continuously applies the pivoting strategy. Let  $r = \bar{d}_C(u)$ , the recurrence for this case is given by  $T(n) \leq T(n-r-1) + \sum_{i=1}^r T(n-r-i)$ . If  $r = 0$ ,  $T(n) \leq T(n-1)$ . If  $r = 1$ ,  $T(n) \leq 2T(n-2)$ . If  $r \geq 2$ , let  $T(n) = \beta^n$  according to the existing theory [17], where  $1 < \beta \leq 2$ . Since  $\beta^{n-r-1} + \sum_{i=1}^r \beta^{n-r-i} = \beta^{n-r-1} + \beta^{n-2r} \frac{\beta^r - 1}{\beta - 1} \leq \beta^{n-2} + \frac{\beta^{n-r}}{\beta - 1} \leq 2\beta^{n-2}$ , we have  $T(n) \leq 2T(n-2)$ . Therefore, the recurrence for this case is given by  $T(n) \leq 2T(n-2)$ .
- (3) Otherwise, let  $u = \arg \min_{v \in C_0} \bar{d}_C(v)$ . If  $\bar{d}_C(u) \leq k - |\bar{E}(S)|$ , BranchP performs the pivoting strategy with  $u$  as the pivot vertex. This strategy generates at most  $\bar{d}_C(u) + 1 \leq k + 1$  sub-instances. Therefore, the recurrence for this case is given by  $T(n) \leq \sum_{i=1}^{k+1} T(n-i)$ . Otherwise, BranchP uses the pivot vertex  $u$  as the branching vertex. In the next-level branching, we have  $|C \setminus C_0| > k - |\bar{E}(S \cup \{u\})|$ , as  $\bar{d}_C(u) > k - |\bar{E}(S)|$ . In this case, the sub-branch (the term  $T(n-1)$ ) will continuously perform the binary branching strategy as discussed in case (1). Consequently, the recurrence for this case is given by  $T(n) \leq \sum_{i=1}^{k+1} T(n-i) + T(n-2k-2)$ . Note that the term  $T(n-2k-2)$  corresponds to a maximum biclique search problem, and we can derive that  $T(n-2k-2) = 2T(n-2k-4)$  as discussed in case (2). Therefore, the final recurrence for this case is given by  $T(n) \leq \sum_{i=1}^{k+1} T(n-i) + 2T(n-2k-4)$ .

Combining the above analysis, the worst-case recurrence is given by

$$T(n) \leq \sum_{i=1}^{k+1} T(n-i) + 2T(n-2k-4). \quad (2)$$

Based on the theorem proposed in [17], we denote by  $T(n) = \beta_k^n$ , where  $\beta_k$  is the largest real root of equation  $x^n = x^{n-1} + x^{n-2} + \dots + x^{n-k-1} + 2x^{n-2k-4}$ . The equation is equivalent to  $x^{2k+5} - 2x^{2k+4} + x^{k+3} - 2x + 2 = 0$ . Consequently, the theorem is proven.  $\square$

**Discussion.** It is worth noting that there exist various pivoting techniques proposed in previous studies [10, 11, 46]. However, our pivoting-based branching strategy is significantly different from all these existing solutions. Compared to the state-of-the-art pivoting-based maximum  $k$ -defective clique algorithm [10], adapting the

special degree reduction rules and pivoting rules designed for maximum vertex problem in traditional graph to our maximum edge problem proves to be quite challenging. Specifically, if the condition  $|C \setminus \bar{N}_S(C)| \geq k$ , as used in [10], replaces line 10 of Algorithm 2, the time complexity of Algorithm 2 will degrade to  $O^*(2^n)$ . Unlike [10], we employ a more refined condition to determine the appropriate branching strategy, which helps keep the time complexity in check. Additionally, compared to the state-of-the-art pivoting-based maximum biplex enumeration algorithms [46], which achieves a time complexity of  $O(m\beta_k'^n)$  with  $\beta_k' = 1.839$  when  $k = 1$ , our algorithm reaches a better time complexity. This improvements underscores the advancement of our proposed pivoting approaches over other state-of-the-art pivoting techniques.

## 4 OPTIMIZATION TECHNIQUES

In this section, we propose various optimization techniques, including graph reduction techniques to eliminate redundant vertices, upper-bounding techniques to prune unnecessary instances, and a heuristic algorithm for identifying an initial large  $k$ -defective biclique. These techniques can be easily integrated into our proposed algorithms in Sec. 3, and can significantly enhance their efficiency.

### 4.1 Graph Reduction Techniques

**Core-based reduction.** With the constraint of size threshold  $\theta$ , vertices in  $G$  with degrees smaller than  $\theta - k$  can be directly pruned from  $G$ . This pruning can be achieved by calculating an  $(\alpha, \beta)$ -core of  $G$  [5], which we define below.

*Definition 7 (( $\alpha, \beta$ )-core).* Given a bipartite graph  $G = (U, V, E)$  and its subgraph  $G' = (U', V', E')$ , if  $d_{V'}(u) \geq \alpha$  for any  $u \in U'$ , and  $d_{U'}(v) \geq \beta$  for any  $v \in V'$ ,  $G'$  is an  $(\alpha, \beta)$ -core of  $G$ .

We define an  $(\alpha, \beta)$ -core as maximal if it is not contained within any other  $(\alpha, \beta)$ -core. Clearly, a defective biclique of  $G$  constitutes an  $(\alpha, \beta)$ -core of  $G$ , as stated in the following lemma.

**LEMMA 3.** Given a bipartite graph  $G$  and a  $k$ -defective biclique  $D$  of  $G$ ,  $D$  is a  $(|V_D| - k, |U_D| - k)$ -core of  $G$ .

Based on Lemma 3, we can conclude that a  $k$ -MDB of  $G$  is contained in the maximal  $(\theta - k, \theta - k)$ -core of  $G$ . Therefore, we can calculate such a maximal core on the original graph to prune vertices with low degrees, which takes  $O(n + m)$  time by applying the algorithm in [3].

**Common-neighbor-based reduction.** For two vertices  $u$  and  $v$  on the same side, let  $cn(u, v)$  (or  $cn_S(u, v)$ ) denote the number of common neighbors of  $u$  and  $v$  in  $G$  (or in  $S$ ). Given a bipartite graph  $G = (U, V, E)$  and a  $k$ -MDB  $D = (U_D, V_D, E_D)$  of  $G$ , every edge  $(u, v) \in E_D$  satisfies that (1)  $d_D(u) \geq \theta - k$ , and (2) for all  $w \in N_D(v)$ ,  $cn_D(u, w) \geq \theta - k$ . By combining the two conditions, we can conclude that for an edge  $(u, v) \in E$ , if  $v$  has fewer than  $\theta - k$  neighbors  $w$  such that  $cn(u, w) \geq \theta - k$ , the edge  $(u, v)$  can be pruned in  $G$ .

Based on this idea, we present Algorithm 3 to prune such edges from  $G$ , referred to as CnReduction. The algorithm enumerates all edges  $(u, v)$  in ascending order of  $u$ 's degree (lines 1-2). For each edges  $(u, v)$ , the algorithm counts common neighbors between  $u$  and the neighbors  $w$  of  $v$  (lines 3-8). If  $v$  has fewer than  $\theta - k$

---

**Algorithm 3:** CnReduction( $G, k, \theta$ )

---

**Input:** Bipartite graph  $G = (U, V, E)$ , integers  $k$  and  $\theta$   
**Output:** A reduced bipartite graph

```

1  $O \leftarrow$  the ascending degree order of  $U \cup V$ ;
2 for  $u \in O$  do
3   for  $v \in N(u)$  do
4     for  $w \in N(v)$  do
5        $cn(w) \leftarrow 0$ ;
6   for  $v \in N(u)$  do
7     for  $w \in N(v)$  do
8        $cn(w) \leftarrow cn(w) + 1$ ;
9   for  $v \in N(u)$  do
10     $N' \leftarrow \{w \in N(v) \mid cn(w) \geq \theta(v) - k\}$ ;
11    if  $|N'| < \theta - k$  then  $E \leftarrow E \setminus (u, v)$ ;
12    if  $d(u) < \theta - k$  then  $U \leftarrow U \setminus u$ ;
13    for  $v \in N(u)$  s.t.  $d(v) < \theta - k$  do  $V \leftarrow V \setminus v$ ;
14 return  $G$ ;
```

---

neighbors  $w$  satisfying  $cn(u, w) \geq \theta - k$ , the edge  $(u, v)$  is pruned from  $G$  (lines 9-11). Additionally, if the degree of  $u$  or  $v$  drop below  $\theta - k$  due to the deletion of  $(u, v)$ , the algorithm will also prune  $u$  or  $v$  from  $G$  (lines 12-13). We give the time complexity of Algorithm 3 in the following lemma.

LEMMA 4. Given a bipartite graph  $G = (U, V, E)$ , the complexity of Algorithm 3 is  $O(d \cdot m)$ , where  $d = \max_{u \in U \cup V} d(u)$ .

In practice, Algorithm 3 performs efficiently although its time complexity is  $O(d \cdot m)$ . This is because in real-world graphs, most vertices have degrees much smaller than  $d$ . As a result, the actual runtime of the algorithm often does not reach this worst-case time complexity.

**One-non-neighbor reduction.** For an instance  $(S, C)$  and a vertex  $u$ , let  $C_u$  denote the subset of  $C$  where all vertices are on the same side as  $u$ . If a branching vertex has only one non-neighbor in  $S \cup C$ , removing it from  $C$  can lead to the pruning of additional vertices. The following lemma explains this idea in detail.

LEMMA 5. Given an instance  $(S, C)$  and a vertex  $u \in C$  with  $\bar{d}_{S \cup C}(u) = 1$ , if  $u$  is removed from  $C$ , all vertices  $v \in C_u$  with  $\bar{d}_S(v) \geq 1$  can be pruned from  $C$ . Moreover, if  $d_C(u) = 1$  and  $w$  is the unique non-neighbor of  $u$ , then all vertices in  $\bar{N}_C(w)$  can also be pruned from  $C$ .

PROOF. Assume  $D$  is a  $k$ -MDB derived from  $(S, C)$  excluding  $u$ . For any  $v \in C_u$  such that  $v \in D$  and  $\bar{d}_S(v) \geq 1$ , we have  $D \setminus \{v\} \cup \{u\}$  is also a  $k$ -MDB. In the case where  $\bar{d}_C(u) = 1$  and  $w$  is the unique non-neighbor of  $u$ , if  $w \notin D$ , then  $D \cup \{u\}$  forms a larger  $k$ -MDB. Otherwise, for any  $v \in \bar{N}_C(w)$  such that  $v \in D$ , we have  $D \setminus \{v\} \cup \{u\}$  is also a  $k$ -MDB.  $\square$

**Ordering-based reduction.** Given a bipartite graph  $G = (U, V, E)$ , let  $u_1, u_2, \dots, u_{|U|}$  be an arbitrary vertex order of  $U$ . For each vertex  $u_i$ , the ordering-based reduction aims to find a  $k$ -MDB  $D_i$  including  $u_i$  while excluding  $u_1, u_2, \dots, u_{i-1}$ . Obviously, the  $k$ -MDB of  $G$  is the one with most edges among  $D_1, D_2, \dots, D_{|U|}$ . Next, we extend Lemma 1 to constrain the vertex distance in  $D_i$ , as shown in the following lemma.

LEMMA 6. Given a  $k$ -defective biclique  $D$ , if  $|U_D| > k$  and  $|V_D| > k$ , the maximum distance between any two vertices of  $D$  is 3.

Lemma 6 indicates that when searching for  $D_i$ , vertices with distances more than 3 from  $u_i$  can be pruned. Let  $N^2(u) = \{w \in N(v) \mid v \in N(u)\}$ , i.e., the set of  $u$ 's 2-hop neighbors. Let  $N_+^2(u_i) = N^2(u_i) \cap \{u_{i+1}, u_{i+2}, u_{|U|}\}$ . According to Lemma 6, we can construct an initial instance  $(S_i, C_i)$  to find  $D_i$ , where  $S_i = (\{u_i\}, \emptyset)$  and  $C_i = (N_+^2(u_i), \bigcup_{u_j \in N_+^2(u_i)} N(u_j))$ .

In practical implementation, to identify a larger  $k$ -defective biclique as early as possible, we traverse the vertices in  $U$  in descending order of their degrees. This approach is based on the intuition that vertices with higher degrees are more likely to be present in an MDB. Additionally, the core-based and common-neighbor-based reduction techniques can be employed on  $(S_i, C_i)$  to prune unnecessary vertices and edges. Specifically, vertices  $u \in C_i$  are pruned if  $d_{S_i \cup C_i}(u) < \theta - k$ , and edges  $(u, v)$  are pruned if there are fewer than  $\theta - k$  neighbors  $w$  of  $v$  satisfying  $cn(u, w) \geq \theta - k$ .

**Progressive bounding reduction.** The size threshold  $\theta$  is provided as an input parameter and can be set to a very small value (e.g.,  $k + 1$ ), which may limit the effectiveness of our optimization techniques. To address this issue, we utilize the progressive bounding technique inspired from the maximum biclique problem [33] to provide tighter size thresholds for the  $k$ -MDB search problem.

Basically, the progressive bounding reduction iteratively runs our algorithm over multiple rounds, with each round providing two size thresholds for both sides. Specifically, let  $\theta_U^t$  and  $\theta_V^t$  denote the size thresholds in  $t$ -th round. The objective of  $t$ -th round is to identify a  $k$ -MDB  $D_t = (U_{D_t}, V_{D_t}, E_{D_t})$  satisfying  $|U_{D_t}| \geq \theta_U^t$  and  $|V_{D_t}| \geq \theta_V^t$ . Let  $D^* = \max\{D_1, D_2, \dots, D_{t-1}\}$  be the currently largest  $k$ -defective biclique. The progressive bounding reduction generates  $\theta_U^t$  and  $\theta_V^t$  in the following way:

$$\theta_U^t = \max\{\theta, \lfloor \theta_U^{t-1} / 2 \rfloor\}, \quad (3)$$

$$\theta_V^t = \max\{\theta, \lfloor |E_{D^*}| / \theta_U^{t-1} \rfloor\}. \quad (4)$$

Specifically,  $\theta_U^0 = \max_{u \in U} d_U(u) + k$ , and the iteration terminates when  $\theta_U^t = 0$ . The following lemma shows the correctness of the progressive bound reduction.

LEMMA 7. Given a bipartite graph  $G$  and a  $k$ -MDB  $D$  of  $G$ , there always exists an integer  $t$  such that  $\theta_U^t \leq |U_D|$  and  $\theta_V^t \leq |V_D|$ .

The proof of Lemma 7 can be easily obtained by applying a similar argument as used in [33]. In many real-world graphs,  $\theta_U^t$  and  $\theta_V^t$  are often significantly larger than  $\theta$ , which helps facilitate more effective graph reduction, such as a smaller reduced graph by performing the core-based reduction and common-neighbor-based reduction.

## 4.2 Novel Upper-Bounding Techniques

In this subsection, We propose a novel upper-bounding technique that provides effective upper bounds for both vertices and edges. Formally, given an instance  $(S, C)$ , assume  $u_1, u_2, \dots, u_{|U_C|}$  and  $v_1, v_2, \dots, v_{|V_C|}$  are the vertices of  $U_C$  and  $V_C$  in ascending order of  $\bar{d}_S(\cdot)$ , respectively. Let  $\bar{d}_S^i(U_C) = \sum_{t=1}^i \bar{d}_S(u_i)$  and  $\bar{d}_S^i(V_C) = \sum_{t=1}^i \bar{d}_S(v_i)$ , i.e., the total number of non-neighbors in  $S$  among the first  $i$  vertices of  $U_C$  and  $V_C$ . Based on this, the vertex upper bound is presented in the following lemma.

---

**Algorithm 4:** UpperBound( $(S, C), k$ )

---

**Input:** Instance  $(S = (U_S, V_S), C = (U_C, V_C))$  and integer  $k$   
**Output:** two vertex upper bounds and an edge upper bound

```
1  $k' \leftarrow k - |\bar{E}_S|;$ 
2 Let  $u_1, u_2, \dots, u_{|U_C|}$  and  $v_1, v_2, \dots, v_{|V_C|}$  be the vertices of  $U_C$  and  $V_C$  in
   ascending order of  $\bar{d}_S(\cdot)$ ;
3  $\bar{d}_U \leftarrow 0; \bar{d}_V \leftarrow 0; c_U \leftarrow 0; c_V \leftarrow 0;$ 
4 for  $i \leftarrow 1 \dots |U_C|$  s.t.  $\bar{d}_U + \bar{d}_S(u_i) \leq k'$  do
5    $\bar{d}_U \leftarrow \bar{d}_U + \bar{d}_S(u_i);$ 
6    $c_U \leftarrow c_U + 1;$ 
7  $e \leftarrow (|U_S| + c_U) \cdot |V_S| - |\bar{E}_S| - \bar{d}_U;$ 
8  $i \leftarrow c_U;$ 
9 for  $j \leftarrow 1 \dots |V_C|$  s.t.  $\bar{d}_V + \bar{d}_S(v_j) \leq k'$  do
10  while  $\bar{d}_U + \bar{d}_V + j > k'$  do
11     $\bar{d}_U \leftarrow \bar{d}_U - \bar{d}_S(u_i);$ 
12     $i \leftarrow i - 1;$ 
13   $\bar{d}_V \leftarrow \bar{d}_V + \bar{d}_S(v_j);$ 
14   $c_V \leftarrow c_V + 1;$ 
15   $e \leftarrow \max\{e, (|U_S| + i) \cdot (|V_S| + j) - |E_S| - \bar{d}_U - \bar{d}_V\};$ 
16 return  $|U_S| + c_U, |V_S| + c_V, e;$ 
```

---

LEMMA 8 (Vertex upper bounds). For any  $k$ -MDB  $D = (U_D, V_D, E_D)$  derived from  $(S, C)$ , we have  $|U_D| \leq |U_S| + i$  and  $|V_D| \leq |V_S| + j$ , where  $i$  and  $j$  are the largest values satisfying  $\bar{d}_S^i(U_C) \leq k - |\bar{E}_S|$  and  $\bar{d}_S^j(V_C) \leq k - |\bar{E}_S|$ , respectively.

PROOF. For the side of  $U$ , it is obvious that  $i$  is the maximum number of vertices that can be added from  $U_C$  to  $U_S$  such that  $(U_S \cup \{u_1, u_2, \dots, u_i\}, V_S)$  remains a valid  $k$ -defective biclique. For any  $k$ -MDB derived from  $(S, C)$  consisting of  $(U_S \cup U_{C'}, V_S \cup V_{C'})$ ,  $(U_S \cup U_{C'}, V_S)$  is still a  $k$ -defective biclique. Therefore, we have that  $|U_{C'}| \leq i$ . The proof on the other side is similar.  $\square$

Based on Lemma 8, if both sides reach their vertex upper bounds, an edge upper bound can be calculated as  $(|U_S| + i) \times (|V_S| + j) - |\bar{E}_S| - \bar{d}_S^i(U_C) - \bar{d}_S^j(V_C)$ . However, this upper bound does not account for the coexistence of vertices in  $U_C$  and  $V_C$ . To solve the issue, we present a tighter edge upper bound in the following lemma.

LEMMA 9 (Edge upper bound). For any  $k$ -MDB  $D$  derived from  $(S, C)$ , we have that  $|E_D| \leq \max_{0 \leq i \leq |U_C|} (|U_S| + i) \times (|V_S| + j) - |\bar{E}_S| - \bar{d}_S^i(U_C) - \bar{d}_S^j(V_C)$ , where  $j$  is the largest value satisfying  $\bar{d}_S^i(U_C) + \bar{d}_S^j(V_C) \leq k - |\bar{E}_S|$ .

PROOF. Consider a new instance  $(S, C')$  reconstructed from  $(S, C)$  by fully connecting vertices between  $U_{C'}$  and  $V_{C'}$ , i.e.,  $\bar{E}_{C'} = \emptyset$ . The ordering of vertices in  $C'$  with respect to  $\bar{d}_S(\cdot)$  remains unchanged. There exists a  $k$ -MDB derived from  $(S, C')$  composed of  $S$ , a prefix of  $\{u_1, u_2, \dots, u_{|U_C|}\}$ , and a prefix of  $\{v_1, v_2, \dots, v_{|V_C|}\}$  (otherwise, for any vertex not in a prefix, a preceding vertex can always be found to replace it). Moreover, the  $k$ -MDB derived from  $(S, C')$  have more edges than any  $k$ -MDB derived from  $(S, C)$ . Consequently, Lemma 9 provide a correct edge upper bound.  $\square$

**Implementation details.** Algorithm 4 presents a pseudocode of our upper-bounding techniques. It starts by setting  $k'$  as the remaining number of missing edges (line 1), and sorting the vertices of  $U_C$  and  $V_C$  in ascending order of  $\bar{d}_S(\cdot)$  (line 2), which can be efficiently done using a bucket sort in linear time and space. Then,

the algorithm calculates the vertex upper bound in  $U$  by finding the longest prefix of  $U_C$  that can be added to  $U_S$  (lines 4-6), where  $c_U$  records the prefix length. After that, the algorithm derives the upper bound  $c_V$  for vertices in  $V$  using the same way, and simultaneously calculates the upper bound  $e$  for edges by removing vertices from the end of the prefix of  $U_C$  while computing the longest prefix of  $V_C$  (lines 9-15). Finally, it returns two upper bounds of both sides as well as an edge upper bound. We present the time complexity of the algorithm in the following lemma.

LEMMA 10. The time complexity of Algorithm 4 is  $O(n)$ .

### 4.3 A Heuristic Algorithm

We propose a heuristic algorithm to efficiently calculate a large  $k$ -defective biclique. We observe that in many real-world graphs, one side of the  $k$ -MDB with size threshold  $\theta$  contains exactly  $\theta$  vertices, while the other side contains much more than  $\theta$  vertices. Motivated by this, our heuristic algorithm aims to find a large  $k$ -defective biclique with one side has  $\theta$  vertices, which is presented in Algorithm 5. The heuristic algorithm attempts to calculate two  $k$ -defective bicliques given by  $(U_0, V_0)$  and  $(U_1, V_1)$ , where  $U_0$  have  $\theta$  vertices and  $V_1$  have  $\theta$  vertices, respectively (lines 1-2). The one with more edges will be returned as the large  $k$ -defective biclique (lines 3-4).

The Expand procedure aims to compute a  $k$ -defective biclique with  $\theta$  vertices on the side of  $U$ . Specifically, it initializes two vertex sets  $U' = \emptyset$  and  $V' = V$  (line 6). The algorithm then iteratively attempts to add  $\theta$  vertices to  $U'$  (lines 7-18). In each iteration, the vertex  $u \in U$  with the highest degree in  $V'$  is selected for addition (line 8). According to the core-based reduction discussed in Sec. 4.1, if  $d_{V'}(u) < \theta - k$ , the iteration terminates early, as no more vertex in  $U$  can be added to  $U'$  (line 9). Then, the algorithm determines whether  $u$  can be added to  $U'$  by ensuring that there are  $\theta$  vertices in  $V'$  that can form a  $k$ -defective biclique with  $U' \cup \{u\}$  (lines 10-13). If the condition is satisfied, the algorithm repeatedly removes vertices from  $V'$  that have the most non-neighbors in  $U' \cup \{u\}$ , until  $u$  can be added to  $U'$  (lines 14-18). Finally, if  $U'$  contains fewer than  $\theta$  vertices, the algorithm returns an empty result. Otherwise, the algorithm yields  $(U', V')$  as a large  $k$ -defective biclique with  $|U'| = \theta$ . Note that finding a vertex with maximum (minimum) degree (lines 8, 12 and 16) can be implemented in constant time using the decomposition techniques similar to [3]. We present the time complexity of Algorithm 5 in the following lemma.

LEMMA 11. Given a bipartite graph  $G = (U, V, E)$  and two integers  $k, \theta$ , the time complexity of Algorithm 5 is  $O(\theta n + m)$ .

### 4.4 Optimized Algorithms

All the proposed optimization techniques can be applied to our branch-and-bound algorithm and pivoting-based algorithm. We refer to the branch-and-bound algorithm and pivoting-based algorithm equipped with all the optimization techniques as MDBB and MDBP, respectively. An implementation of MDBP is presented in Algorithm 6. The algorithm begins by obtaining an initial  $k$ -defective biclique (line 1) by invoking the heuristic algorithm (Algorithm 5). Then it starts to find  $k$ -MDB by performing the progressive bounding reduction (lines 2-5). In each iteration, two size thresholds  $\theta_U$  and  $\theta_V$  are generated based on Eq. (3) and Eq. (4). Then,

---

**Algorithm 5: Heuristic( $G, k, \theta$ )**


---

**Input:** Bipartite graph  $G = (U, V, E)$ , integers  $k$  and  $\theta$   
**Output:** A large  $k$ -defective biclique

```

1  $(U_0, V_0) \leftarrow \text{Expand}(U, V)$ ;
2  $(V_1, U_1) \leftarrow \text{Expand}(V, U)$ ;
3 if  $|E(U_0, V_0)| \geq |E(U_1, V_1)|$  then return  $G(U_0, V_0)$ ;
4 else return  $G(U_1, V_1)$ ;
5 Function  $\text{Expand}(U, V)$ :
6    $U' \leftarrow \emptyset$ ;  $V' \leftarrow V$ ;
7   while  $|U'| < \theta$  do
8      $u \leftarrow \arg \max_{u \in U \setminus U'} d_{V'}(u)$ ;
9     if  $d_{V'}(u) + k < \theta$  then break;
10     $\bar{d} \leftarrow 0$ ;
11    for  $i$  in  $1 \dots \theta$  do
12       $v \leftarrow \arg \min_{v \in V' \setminus \bar{d}_{U' \cup \{u\}}(v)}$ ;
13       $\bar{d} \leftarrow \bar{d} + \bar{d}_{U' \cup \{u\}}(v)$ ;
14    if  $\bar{d} \leq k$  then
15      while  $|\bar{E}(U', V')| + \bar{d}_{V'}(u) > k$  do
16         $v \leftarrow \arg \max_{v \in V' \setminus \bar{d}_{U' \cup \{u\}}(v)}$ ;
17         $V' \leftarrow V' \setminus \{v\}$ ;
18       $U' \leftarrow U' \cup \{u\}$ ;
19 if  $|U'| < \theta$  then return  $(\emptyset, \emptyset)$ ;
20 else return  $(U', V')$ ;
```

---

the core-based reduction and common-neighbor-based reduction are employed on the input graph  $G$  by invoking  $\text{ReduceG}$  (line 6), which involves calculating a  $(\theta_V - k, \theta_U - k)$ -core of  $G$  (line 15) and invoking  $\text{CnReduction}$  (Algorithm 3) on the  $(\theta_V - k, \theta_U - k)$ -core (line 16). the algorithm continues by performing the ordering-based reduction on the reduced graph (lines 7-12). In detail, it traverses each vertex  $u$  of  $U$  in descending degree order (lines 7-8), and constructs an initial instance  $(S, C)$ . After that, the algorithm further reduce the size of  $C$  by invoking  $\text{ReduceI}$  (line 11), which utilizes the core-based reduction and common-neighbor-based reduction again (lines 19-20). Finally, the algorithm runs  $\text{BranchP}^*$  to find  $k$ -MDB in  $(S, C)$ , which is adapted from  $\text{BranchP}$  in Algorithm 2 by additionally implementing the graph reduction techniques and the upper-bounding techniques (Algorithm 4). By taking the maximum returned solution from all invocations of  $\text{BranchP}^*$ , the algorithm derives a  $k$ -MDB  $D^*$  at the end and returns  $D^*$  as the final answer (line 13). The implementation of MDBB is quite similar to that of MDBP, which differs only in that replacing  $\text{BranchP}^*$  in Algorithm 6 with  $\text{BranchB}$  from Algorithm 1 incorporating the optimization techniques used in  $\text{BranchP}^*$ .

## 5 EXPERIMENTS

### 5.1 Experimental Setup

**Datasets.** We collect 10 real-world bipartite graphs from diverse categories to evaluate the efficiency of our algorithms. These graphs have been widely used as benchmark datasets for testing other bipartite subgraph search problem [13, 33, 46]. The detailed information of these graphs are shown in Table 1, where  $\max d_U$  and  $\max d_V$  represent the maximum degree of vertices in  $U$  and  $V$ , respectively. All the datasets are available at <http://konect.cc>.

**Algorithms.** We implement two proposed algorithm MDBB and MDBP, which are respectively the branch-and-bound algorithm in Algorithm 1 and the pivoting-based algorithm in Algorithm 2 equipped with all our proposed optimization techniques in Sec. 4. We implement a baseline algorithm referred to as MDCadp for

---

**Algorithm 6: MDBP**


---

**Input:** A bipartite graph  $G = (U, V, E)$ , integers  $k, \theta$   
**Output:** A  $k$ -MDB of  $G$

```

1  $D^* \leftarrow \text{Heuristic}(G, k, \theta)$ ;
2  $\theta_U \leftarrow \max_{u \in U} d(u)$ ;
3 while  $\theta_U > \theta$  do
4    $\theta_V \leftarrow \max\{\theta, \lfloor |D^*|/\theta_U \rfloor\}$ ;  $\theta_U \leftarrow \max\{\theta, \lfloor \theta_U/2 \rfloor\}$ ;
5    $G' \leftarrow \text{ReduceG}(G, k, \theta_U, \theta_V)$ ;
6    $\{u_1, u_2, \dots, u_{|U|}\} \leftarrow$  the descending degree order of  $U$ ;
7   for  $i \in 1 \dots |U|$  do
8      $S \leftarrow (\{u_i\}, \emptyset)$ ;  $C \leftarrow (N_+^2(u_i), \bigcup_{v \in N_+^2(u_i)} N(v))$ ;
9      $C \leftarrow \text{ReduceI}(C, u_i, \theta_U, \theta_V)$ ;
10     $\text{BranchP}^*(S, C)$ ; // BranchP equipped with graph reduction
    techniques and upper-bounding techniques
11 return  $D^*$ ;
12 Function  $\text{ReduceG}(G, k, \theta_U, \theta_V)$ :
13    $G' \leftarrow (\theta_V - k, \theta_U - k)$ -core of  $G$ ;
14    $G' \leftarrow \text{CnReduction}(G', k, \min\{\theta_U, \theta_V\})$ ;
15   return  $G'$ ;
16 Function  $\text{ReduceI}(C = (U_C, V_C), u, \theta_U, \theta_V)$ :
17    $U_{C'} \leftarrow \{v \in U_C \mid \text{cn}(u, v) \geq \theta_V - k\}$ ;
18    $V_{C'} \leftarrow \{v \in V_C \mid d(v) \geq \theta_U - k\}$ ;
19   return  $(U_{C'}, V_{C'})$ 
```

---

**Table 1: Datasets used in experiments**

| Dataset | $ U $     | $ V $     | $ E $      | $\max d_U$ | $\max d_V$ |
|---------|-----------|-----------|------------|------------|------------|
| LKML    | 42,045    | 337,509   | 599,858    | 31,719     | 6,627      |
| Team    | 901,130   | 34,461    | 1,366,466  | 17         | 2,671      |
| Mummun  | 175,214   | 530,418   | 1,890,661  | 968        | 19,805     |
| Citeu   | 153,277   | 731,769   | 2,338,554  | 189,292    | 1,264      |
| IMDB    | 303,617   | 896,302   | 3,782,463  | 1,334      | 1,590      |
| Enwiki  | 18,038    | 2,190,091 | 4,129,231  | 324,254    | 1,127      |
| Amazon  | 2,146,057 | 1,230,915 | 5,743,258  | 12,180     | 3,096      |
| Twitter | 244,537   | 9,129,669 | 10,214,177 | 640        | 18,874     |
| Aol     | 4,811,647 | 1,632,788 | 10,741,953 | 100,629    | 84,530     |
| Google  | 5,998,790 | 4,443,631 | 20,592,962 | 423        | 95,165     |

comparison. MDCadp is adapted from the state-of-the-art maximal defective clique algorithm  $\text{Pivot+}$  [12]. Specifically, MDCadp transfers the input bipartite graph into a traditional graph by treating all vertices on the same side as fully connected, and performs the optimization techniques we presented in Sec. 4.1 and 4.3. The outcome of MDCadp is obtained by filtering out a  $k$ -MDB from all the maximal defective cliques of the transferred graph. All the algorithms are implemented in C++, and run on a PC with a 2.2GHz CPU and 128GB memory running Linux.

**Parameter settings.** During the evaluations of all algorithms, we vary the value of  $k$  from 1 to 6. As outlined in Sec. 2, we focus on finding a connected  $k$ -MDB, achieved by setting  $\theta > k$ . Based on this, we vary the value of  $\theta$  from  $k + 1$  to 10 for a specified  $k$ .

### 5.2 Performance Studies

**Exp1: Efficiency comparison among different algorithms.** We evaluate the efficiency of two proposed algorithms MDBB, MDBP and the baseline algorithm MDCadp. The results are shown in Table 2, where the column  $|E^*|$  represents the number of edges in the  $k$ -MDB, and the notion “-” represents a timeout result that cannot finish calculating within 3 hours. As observed, our proposed algorithm MDBP achieves the best performance for most of the input parameters. Specifically, compared to the baseline algorithm

Table 2: Results of runtime among different algorithms on 10 real-world graphs (in seconds)

| Dataset | $k$ | $\theta$ | $ E^* $       | MDBP            | MDBB          | MDCadp | $k$ | $\theta$ | $ E^* $       | MDBP            | MDBB     | MDCadp | $k$ | $\theta$ | $ E^* $         | MDBP            | MDBB     | MDCadp |
|---------|-----|----------|---------------|-----------------|---------------|--------|-----|----------|---------------|-----------------|----------|--------|-----|----------|-----------------|-----------------|----------|--------|
| LKML    | 1   | 2        | 1,781         | <b>13.03</b>    | 17.08         | -      | 2   | 3        | 460           | <b>43.72</b>    | -        | -      | 3   | 4        | 149             | <b>267.01</b>   | -        | -      |
|         | 5   | 84       | <b>1.61</b>   | 2,146           | 28.19         | -      | 6   | 70       | <b>7.603</b>  | 30.14           | -        | -      | 7   | 60       | <b>9.53</b>     | 37.96           | -        | -      |
|         | 4   | 7        | 66            | <b>103.03</b>   | 1,216.70      | -      | 5   | 8        | 67            | <b>107.31</b>   | 1,224.86 | -      | 6   | 9        | 0               | <b>248.95</b>   | 1,732.26 | -      |
| Team    | 1   | 2        | 701           | <b>19.26</b>    | 39.98         | -      | 2   | 3        | 265           | <b>45.51</b>    | -        | -      | 3   | 4        | 149             | <b>154.56</b>   | -        | -      |
|         | 5   | 99       | <b>7.14</b>   | 7.19            | 66.18         | -      | 6   | 64       | <b>9.53</b>   | 14.09           | -        | -      | 7   | 0        | <b>10.07</b>    | 15.41           | -        | -      |
|         | 4   | 5        | 111           | <b>591.71</b>   | -             | -      | 5   | 6        | 79            | <b>2684.57</b>  | -        | -      | 6   | 8        | 0               | <b>927.43</b>   | -        | -      |
| Mummun  | 1   | 2        | 10,315        | <b>18.49</b>    | 18.62         | -      | 2   | 3        | 8,803         | <b>22.88</b>    | 8,661.39 | -      | 3   | 4        | 4,185           | <b>39.95</b>    | -        | -      |
|         | 5   | 2,469    | <b>13.57</b>  | 13.59           | -             | -      | 6   | 1,768    | <b>21.08</b>  | 37.83           | -        | -      | 7   | 1,145    | <b>33.58</b>    | 916.16          | -        | -      |
|         | 4   | 5        | 2,481         | <b>45.73</b>    | -             | -      | 5   | 6        | 1,783         | <b>69.39</b>    | -        | -      | 6   | 7        | 1,163           | <b>160.89</b>   | -        | -      |
| Citeu   | 1   | 2        | 12,425        | <b>141.21</b>   | 146.81        | -      | 2   | 3        | 12,427        | <b>148.38</b>   | 187.69   | -      | 3   | 4        | 9,989           | <b>137.15</b>   | 4,332.55 | -      |
|         | 5   | 7,394    | <b>5.48</b>   | 5.69            | -             | -      | 6   | 4,960    | <b>7.88</b>   | 77.20           | -        | -      | 7   | 3,063    | <b>17.92</b>    | 3,500.77        | -        | -      |
|         | 4   | 5        | 7,406         | <b>141.78</b>   | -             | -      | 5   | 6        | 4,975         | <b>158.43</b>   | -        | -      | 6   | 7        | 3,081           | <b>253.15</b>   | -        | -      |
| IMDB    | 1   | 2        | 775           | 6.99            | <b>5.39</b>   | -      | 2   | 3        | 562           | <b>8.86</b>     | 16.96    | -      | 3   | 4        | 522             | <b>15.85</b>    | 436.35   | -      |
|         | 5   | 514      | 6.12          | <b>5.35</b>     | 23.43         | -      | 6   | 514      | <b>6.76</b>   | 6.91            | 1,123.22 | -      | 7   | 482      | <b>14.73</b>    | 25.23           | -        | -      |
|         | 4   | 5        | 526           | <b>19.99</b>    | 593.86        | -      | 5   | 6        | 529           | <b>31.05</b>    | 683.75   | -      | 6   | 7        | 498             | <b>57.45</b>    | 7,936.24 | -      |
| Enwiki  | 1   | 2        | 203,569       | <b>3,630.07</b> | -             | -      | 2   | 3        | 133,759       | <b>4,977.99</b> | -        | -      | 3   | 4        | 80,461          | <b>3,259.88</b> | -        | -      |
|         | 5   | 21,939   | <b>156.70</b> | 158.87          | 387.15        | -      | 6   | 4,240    | <b>199.76</b> | 582.63          | -        | -      | 7   | 3,224    | <b>264.30</b>   | -               | -        | -      |
|         | 4   | 5        | 21,951        | <b>3,826.16</b> | -             | -      | 5   | 6        | 4,255         | <b>5,744.46</b> | -        | -      | 6   | 8        | 2,330           | <b>6,557.63</b> | -        | -      |
| Amazon  | 1   | 2        | 1,319         | <b>42.68</b>    | 45.42         | -      | 2   | 3        | 427           | <b>56.54</b>    | 646.44   | -      | 3   | 4        | 423             | <b>88.75</b>    | 1,195.66 | -      |
|         | 5   | 413      | 11.71         | <b>11.12</b>    | 49.70         | -      | 6   | 418      | 12.91         | <b>12.00</b>    | -        | -      | 7   | 357      | <b>13.15</b>    | 14.91           | -        | -      |
|         | 4   | 5        | 428           | <b>99.80</b>    | 850.49        | -      | 5   | 6        | 433           | <b>249.97</b>   | -        | -      | 6   | 7        | 390             | <b>268.21</b>   | -        | -      |
| Twitter | 1   | 2        | 5,369         | <b>11.13</b>    | 13.24         | -      | 2   | 3        | 5,383         | <b>10.88</b>    | 51.42    | -      | 3   | 4        | 5,397           | <b>15.47</b>    | 320.65   | -      |
|         | 5   | 5,369    | <b>6.27</b>   | 7.60            | 8,976.29      | -      | 6   | 5,383    | <b>6.63</b>   | 19.51           | -        | -      | 7   | 5,397    | <b>9.76</b>     | 84.34           | -        | -      |
|         | 4   | 5        | 5,411         | <b>20.75</b>    | 3,549.73      | -      | 5   | 6        | 5,425         | <b>31.20</b>    | -        | -      | 6   | 7        | 5,439           | <b>57.67</b>    | -        | -      |
| Aol     | 1   | 2        | 10,869        | 1,032.95        | <b>970.79</b> | -      | 2   | 3        | 3,208         | <b>1,635.79</b> | -        | -      | 3   | 4        | 853             | <b>3,122.56</b> | -        | -      |
|         | 5   | 334      | 49.77         | <b>48.69</b>    | 5,703.42      | -      | 6   | 238      | <b>94.15</b>  | 181.85          | -        | -      | 7   | 207      | <b>123.04</b>   | 408.97          | -        | -      |
|         | 4   | 5        | 346           | <b>9,271.83</b> | -             | -      | 5   | 7        | 219           | <b>2388.21</b>  | -        | -      | 6   | 9        | 174             | -               | -        | -      |
| Google  | 1   | 2        | 10,829        | <b>2,163.49</b> | 2,525.77      | -      | 2   | 3        | 3,370         | <b>4,496.57</b> | 5,115.84 | -      | 3   | 4        | 1,201           | <b>6,250.09</b> | -        | -      |
|         | 5   | 439      | <b>552.42</b> | 573.02          | -             | -      | 6   | 184      | <b>898.04</b> | 2,771.57        | -        | -      | 7   | 151      | <b>1,351.14</b> | -               | -        | -      |
|         | 4   | 7        | 157           | <b>5,331.22</b> | -             | -      | 5   | 8        | 155           | <b>6750.40</b>  | -        | -      | 6   | 9        | 156             | <b>8756.03</b>  | -        | -      |
|         |     | 9        | 140           | <b>613.17</b>   | 9,387.13      | -      | 10  | 145      | <b>683.84</b> | -               | -        | -      | 10  | 145      | <b>1,978.33</b> | -               | -        | -      |

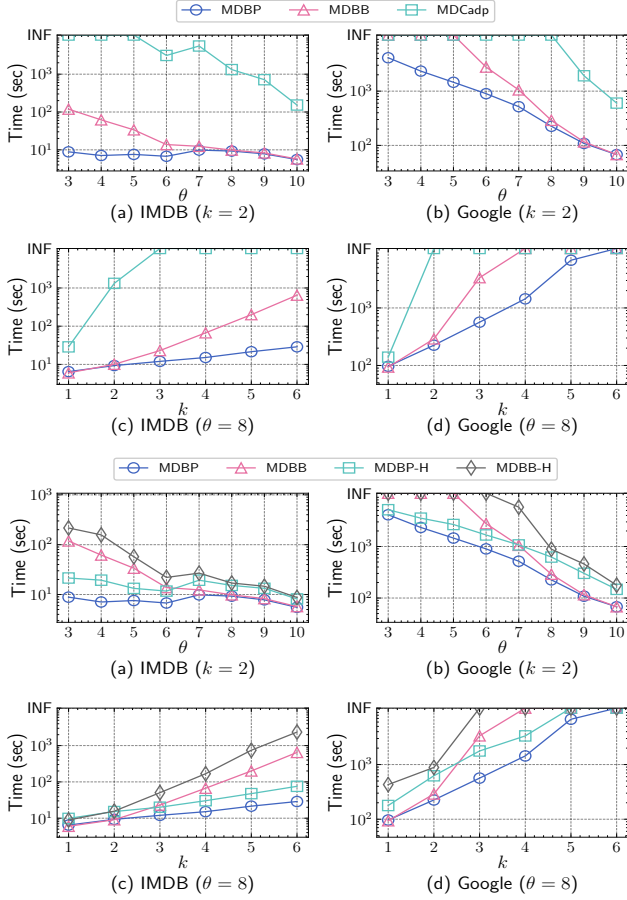
MDCadp, MDBP is at least two order of magnitude faster on most datasets. Furthermore, MDBP essentially solves all tests on most of the graphs when setting  $k \geq 3$ , while MDCadp is almost unable to finish any tests within the time limit. For example, when testing on *Twitter* with  $k = 1$  and  $\theta = 5$ , MDCadp takes 8,976 seconds while MDBP completes within only 6 seconds, achieving a speed up of over 1000 $\times$ . Compared to MDBB, when  $k \leq 2$  and  $\theta \geq 5$ , the efficiency of MDBP is comparable with that of MDBB. However, when  $k \geq 3$  or  $\theta \leq 4$ , MDBP demonstrates a significant advantage. For example, on *Twitter* with  $k = 1$  and  $\theta = 5$ , both MDBP and MDBB runs for about 7 seconds, while on the same graph with  $k = 4$  and  $\theta = 5$ , MDBB takes about 1 hour while MDBP consumes about 20 seconds, achieving more than 150 $\times$  speedup. These results highlights the advancement and high efficiency of our proposed pivoting-based branching strategy. Moreover, when comparing MDBB with MDCadp, we can clearly observe that MDBB also has advantages over MDCadp, which indicates the effectiveness of our proposed new binary branching strategy employed in MDBB.

**Exp2: Results with varying  $\theta$  and  $k$ .** We further evaluate the performance trend of three algorithms MDBB, MDBP and MDCadp with varying  $\theta$  and  $k$  on two representative datasets, *IMDB* and *Google*. The results are shown in Fig. 4, where the notion “INF” represents the runtime exceeds 3 hours. As can be seen, both MDBP and MDBB outperform the baseline algorithm MDCadp with varying  $\theta$  and  $k$ , achieving up to 1000 $\times$  speedups. For example, in Fig. 4(c), MDBP

and MDBB complete in 10 seconds when  $k = 3$ , while MDCadp fails to finish within 3 hours. When comparing MDBP and MDBB, both of them have close runtime when  $\theta \geq 8$  or  $k \leq 2$ , as our optimization techniques significantly reduces the graph size, which highlights the effectiveness of our optimization techniques. Additionally, we find that when  $k$  increases or  $\theta$  decreases, the time curve of MDBP exhibits a gentler slope compared to the other two algorithms, indicating that MDBP is less affected by the input parameters and demonstrates notable scalability.

**Exp3: Runtime of algorithms without using the heuristic algorithm.** To evaluate the effectiveness of our proposed heuristic algorithm, we compare MDBB and MDBP with their versions without the heuristic algorithm (Algorithm 5), referred to as MDBB-H and MDBP-H, respectively. The results are shown in Fig. 5. As observed, MDBB and MDBP always outperform MDBB-H and MDBP-H, respectively. This is because the heuristic algorithm provides a relatively large  $k$ -defective biclique at the beginning, which enhances the effectiveness of our upper-bounding techniques. Additionally, we observe that MDBP-H still always performs better than MDBB when  $\theta \leq 6$  or  $k \geq 3$ , which further highlights the high efficiency of our pivoting techniques.

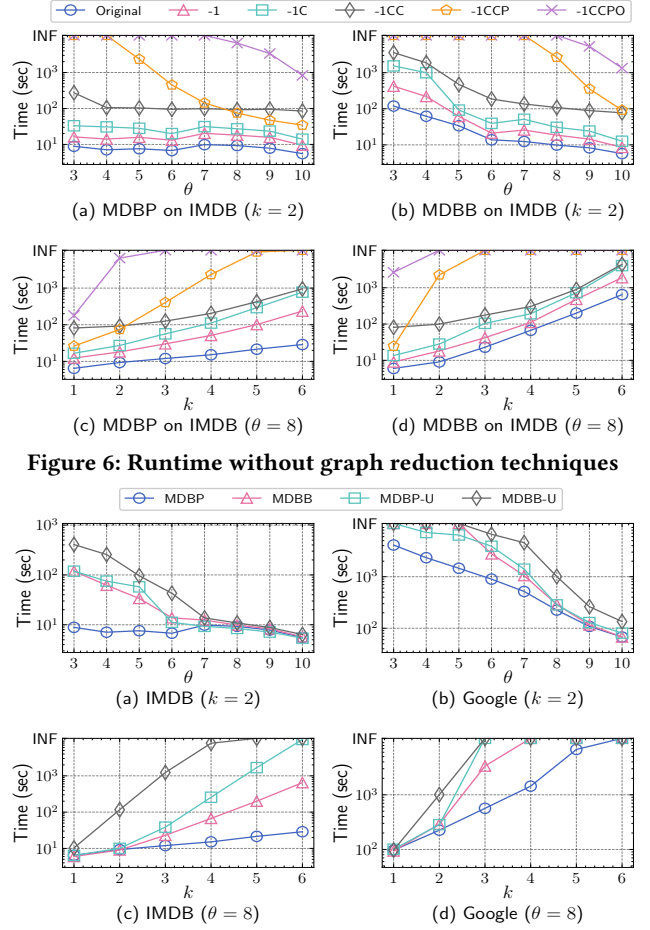
**Exp4: Runtime of algorithms without graph reduction techniques.** To evaluate the effectiveness of our graph reduction techniques, we implement five ablation versions of MDBP and MDBB



**Figure 5: Runtime without using the heuristic algorithm**

by successively removing one-non-neighbor reduction, common-neighbor-based reduction, core-based reduction, progressive bounding reduction and ordering-based reduction from the original algorithm. The results are presented in Fig 6, where the label “Original” represents the original algorithm, and the labels “-1”, “-1C”, “-1CC”, “-1CCP”, “-1CCPO” correspond to the five ablation versions. As observed, all graph reduction techniques contribute to varying degrees of efficiency improvements. Specifically, the one-non-neighbor, core-based and common-neighbor-based reduction techniques provide our algorithms with relatively stable performance improvements with varying  $\theta$  and  $k$ , while the progressive bounding and ordering-based reduction demonstrate a more pronounced improvement when  $\theta$  decreases or  $k$  increases. Notably, when  $k = 1$  or  $\theta = 10$ , the versions of MDBB and MDBP without progressive bounding reduction still outperforms the versions with it. This suggests that our algorithms are well-designed, achieving strong performance for small  $k$  or large  $\theta$  even before the progressive bounding reduction is applied.

**Exp5: Runtime of algorithms without upper-bounding techniques.** To evaluate the effectiveness of our proposed upper-bounding techniques, we compare MDBB and MDBP with their versions excluding the upper-bounding techniques (Algorithm 4), denoted as MDBB-H and MDBP-H, respectively. The results are shown in Fig. 7.



**Figure 6: Runtime without graph reduction techniques**

**Figure 7: Runtime without upper-bounding techniques**

As observed, incorporating the upper-bounding techniques significantly improves the efficiency of both MDBB and MDBP. Moreover, the improvement becomes more pronounced as  $k$  increases or  $\theta$  decreases, indicating the strong effectiveness and efficiency of our proposed upper-bounding techniques. Additionally, the improvement of MDBP by performing upper-bounding techniques is more substantial compared to MDBB. This is likely because the pivoting-based branching strategy introduces more non-edges to the partial set  $S$  earlier, resulting in tighter vertex and edge upper bounds derived from the upper-bounding techniques, which helps prune more unnecessary instances.

**Exp6: Scalability testing.** To evaluate the scalability of algorithms, we test the runtime of MDBB, MDBP and the baseline algorithm MDCadp on *Google* with random sampling of vertices or edges. Results on the other datasets are consistent. We prepare 8 proportionally sampled subgraphs of *Google* by sampling 20%, 40%, 60% and 80% vertices or edges from the original graph. The results are shown in Fig. 8. As can be seen, when the size increases, MDBP shows the slowest growth on runtime, MDCadp exhibits the steepest rise, and MDBB falling in between. This result indicates that when the size of graph increases, MDBP achieves the best scalability among the three algorithms. Furthermore, regardless of graph size, both MDBP and MDBB significantly outperform MDCadp, which further highlights the

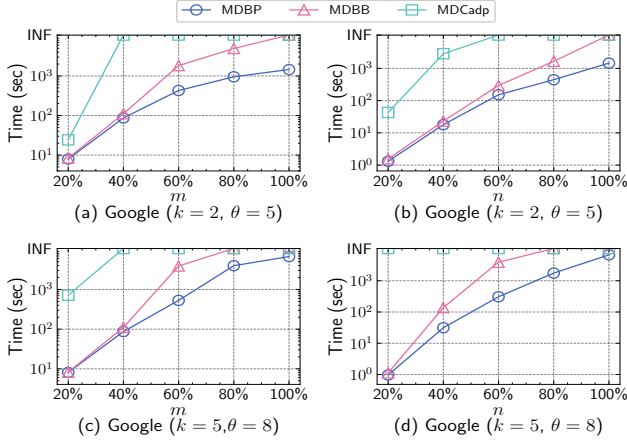


Figure 8: Scalability of different algorithms

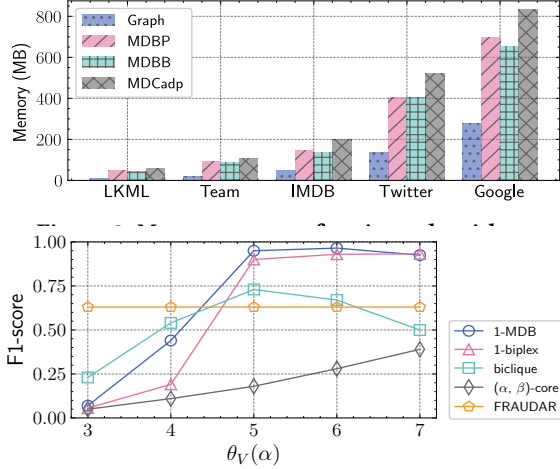


Figure 10: F1-score on fraud detection task

superiority of our proposed branching strategies and optimization techniques.

**Exp7: Memory usage.** In this experiment, we evaluate the memory usage of algorithms MDBB, MDBP and MDCadp on five representative graphs. We measure the maximum physical memory (in MB) used by these algorithms with the parameters  $k = 2$  and  $\theta = 3$ . Similar results can also be observed when using other parameters. The results are shown in Fig. 9. As observed, all three algorithms exhibit comparable space consumption, which is approximately proportional to the size of input graph. Notably, the space consumed by MDCadp is slightly higher than our proposed algorithms. This is mainly because MDCadp requires to fully connect vertices on both sides of a bipartite graph, resulting in increased graph density and a larger search space. These results indicate that our algorithms are space efficient.

### 5.3 Case Study

We conduct a case study to evaluate the effectiveness of  $k$ -MDB in a fraud detection task, following the experimental setup described in [23]. We simulate a camouflage attack scenario on the dataset *Appliance* (<https://amazon-reviews-2023.github.io/>), which contains

602,777 reviews on 30,459 products by 515,650 users. We inject a fraud block consisting of 500 fake users, 500 fake products, along with 10,000 fake reviews by randomly connect fake users and fake products, and 10,000 camouflage reviews by randomly connect fake users and real products.

We adapt MDBP using the approach outlined in [46] to search for the top- $K$  largest  $k$ -MDBs, and execute the adapted algorithm to identify the top 2000  $k$ -MDBs on the attacked graph, treating all users and products in these  $k$ -MDBs as fake. For comparison, we implement four baseline methods: maximum biclique [33], maximum  $k$ -biplex [46] and  $(\alpha, \beta)$ -core, and a widely used fraud detection method, FRAUDAR [23]. To ensure fairness, we identify the top 2000 largest maximum biclique and maximum  $k$ -biplex as fake items. Let  $\theta_U$  and  $\theta_V$  denote the size thresholds of users and products, respectively. The F1-score ( $\frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$ ) results are shown in Fig. 10, where  $\theta_U = \alpha = 3$ . As illustrated, when  $\theta_V \geq 5$ , 1-MDB consistently outperforms other methods. Additionally, 1-MDB surpasses 1-biplex across all values of  $\theta_V$ , likely due to 1-biplex containing more camouflage reviews, resulting in lower precision. For  $\theta_V \leq 4$ , biclique and FRAUDAR outperform both 1-MDB and 1-biplex, as the latter two are too sparse and skewed to provide meaningful structure. However, the F1-score of biclique declines when  $\theta_V \geq 5$ , as the strict definition of biclique limits its ability to detect fake items, resulting in lower recall. In contrast, 1-MDB and 1-biplex, with their more relaxed definitions, still maintain high F1-scores. Overall, these results demonstrate the high effectiveness of our solutions for fraud detection applications.

## 6 RELATED WORK

**Maximum  $k$ -defective clique search.** As a problem related to our work, maximum  $k$ -defective biclique search has received considerable attention in recent years, resulting in the development of many solutions and algorithms [6, 7, 9, 10, 18, 21, 24, 32, 39, 41]. Specifically, Trukhanov et al. [41] proposed the first exact algorithm for detecting a maximum  $k$ -defective clique with a Russian Doll search scheme. Subsequently, Gschwind et al. [21] improved the algorithm with a new preprocessing technique and a better verification procedure. These algorithms are derived from more general clique relaxation search frameworks. There are many studies focusing on the maximum  $k$ -defective clique. Particularly, Chen et al. [9] proposed a branch-and-bound algorithm with a time complexity below  $O^*(2^n)$ , and introduced a coloring-based upper bound to further improve the efficiency. Gao et al. [18] proposed another branch-and-bound algorithm with new vertex and edge reduction rules, achieving better practical performance. Chang [6] proposed an algorithm using a non-fully-adjacent-first branching rule, achieving tighter time complexity than the solutions in [9]. Subsequently, Chang [7] further enhanced the algorithm with an ordering technique, resulting in better time complexity and experimental results. Dai et al. [10] proposed a new branch-and-bound algorithm equipped with a pivot-based branching technique and a series of optimization techniques, attaining the best time complexity and experimental results to date. Nevertheless, it is difficult to adapt these algorithms for the  $k$ -MDB search problem, which is primarily due to the different definitions of maximum.

**Maximum biclique search.** The maximum biclique search problem has been extensively studied in recent years [8, 14, 16, 20, 33, 35, 37, 40, 47]. These studies can be classified into three categories. The first category is maximum edge biclique search, which aims to find a biclique with maximum number of edges. Peeters et al. [35] proved the NP-hardness of the problem, and Ambühl et al. [2] showed the inapproximability of the problem. Shaham et al. [37] proposed a probabilistic algorithm for maximum edge biclique search utilizing a Monte-Carlo subspace clustering method. Shahinpour et al. [38] provided an integer programming method with a scale reduction technique. Lyu et al. [33] proposed a branch-and-bound algorithm with a progressive bounding framework and several graph reduction methods, achieving maximum edge biclique search on billion-scale graphs. The second category is maximum vertex biclique search, which aims to identify a biclique with maximum number of vertices. Garey et al. [19] shown that a maximum vertex biclique can be found in polynomial time using a minimum-cut algorithm. The third category is maximum balanced biclique search, which intends to find a maximum biclique with an equal number of vertices on both sides. Yuan et al. [47] proposed an evolutionary algorithm for the problem with structure mutation. Wang et al. [44] proposed a local search framework with four optimization heuristics. Zhou et al. [48] proposed an upper bound propagation served for two algorithms, which are respectively based on branch-and-bound and integer linear programming. Chen et al. [8] introduced two efficient branch-and-bound algorithms utilizing a bipartite sparsity measurement, targeting small dense and large sparse graphs, achieving improved time complexities and better practical performance. All these mentioned techniques are tailored for maximum biclique search and cannot be directly used to solve our  $k$ -MDB search problem.

## 7 CONCLUSION

In this paper, we investigate the problem of finding a maximum  $k$ -defective biclique in a bipartite graph. We propose two novel algorithms to tackle the problem. The first algorithm employs a newly-designed binary branching strategy that reduces the time complexity to below  $O^*(2^n)$ , while the second algorithm utilizes a novel pivoting-based branching strategy, achieving a notably lower time complexity. Additionally, we propose a series of non-trivial optimization techniques to further improve the efficiency of our algorithms. Extensive experiments conducted on 10 real-word graphs demonstrate the notable effectiveness and efficiency of our algorithms and optimization techniques.

## REFERENCES

- [1] Taher Alzahrani and Kathy Horadam. 2019. Finding maximal bicliques in bipartite networks using node similarity. *Applied Network Science* 4, 1 (2019), 21.
- [2] Christoph Ambühl, Monaldo Mastrolilli, and Ola Svensson. 2011. Inapproximability results for maximum edge biclique, minimum linear arrangement, and sparsest cut. *SIAM J. Comput.* 40, 2 (2011), 567–596.
- [3] Vladimir Batagelj and Matjaz Zaversnik. 2003. An  $O(m)$  algorithm for cores decomposition of networks. *arXiv preprint cs/0310049* (2003).
- [4] Fabian Braun, Olivier Caelen, Evgueni N. Smirnov, Steven Kelk, and Bertrand Lebiclot. 2017. Improving Card Fraud Detection Through Suspicious Pattern Discovery. In *Advances in Artificial Intelligence: From Theory to Practice*, Salem Benferhat, Karim Tabia, and Moonis Ali (Eds.). Springer International Publishing, 181–190.
- [5] Monika Cerinšek and Vladimir Batagelj. 2015. Generalized two-mode cores. *Social Networks* 42 (2015), 80–87.
- [6] Lijun Chang. 2023. Efficient Maximum  $k$ -Defective Clique Computation with Improved Time Complexity. *Proceedings of the ACM on Management of Data* 1, 3 (2023), 1–26.
- [7] Lijun Chang. 2024. Maximum Defective Clique Computation: Improved Time Complexities and Practical Performance. *arXiv preprint arXiv:2403.07561* (2024).
- [8] Lu Chen, Chengfei Liu, Rui Zhou, Jiajie Xu, and Jianxin Li. 2021. Efficient exact algorithms for maximum balanced biclique search in bipartite graphs. In *Proceedings of the 2021 International Conference on Management of Data*. 248–260.
- [9] Xiaoyu Chen, Yi Zhou, Jin-Kao Hao, and Mingyu Xiao. 2021. Computing maximum  $k$ -defective cliques in massive graphs. *Computers & Operations Research* 127 (2021), 105131.
- [10] Qiangqiang Dai, Ronghua Li, Donghang Cui, and Guoren Wang. 2024. Theoretically and Practically Efficient Maximum Defective Clique Search. *Proceedings of the ACM on Management of Data* 2, 4 (2024), 1–27.
- [11] Qiangqiang Dai, Rong-Hua Li, Donghang Cui, Meihao Liao, Yu-Xuan Qiu, and Guoren Wang. 2024. Efficient Maximal Biplex Enumerations with Improved Worst-Case Time Guarantee. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–26.
- [12] Qiangqiang Dai, Rong-Hua Li, Meihao Liao, and Guoren Wang. 2023. Maximal defective clique enumeration. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–26.
- [13] Qiangqiang Dai, Rong-Hua Li, Xiaowei Ye, Meihao Liao, Weipeng Zhang, and Guoren Wang. 2023. Hereditary Cohesive Subgraphs Enumeration on Bipartite Graphs: The Power of Pivot-based Approaches. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–26.
- [14] Milind Dawande, Pinar Keskinocak, Jayashankar M. Swaminathan, and Sridhar R. Tayur. 2001. On Bipartite and Multipartite Clique Problems. *J. Algorithms* 41, 2 (2001), 388–403.
- [15] Kemal Eren, Mehmet Deveci, Onur Küçüktunç, and Ümit V Çatalyürek. 2013. A comparative analysis of biclustering algorithms for gene expression data. *Briefings in bioinformatics* 14, 3 (2013), 279–292.
- [16] Qilong Feng, Shaohua Li, Zeyang Zhou, and Jianxin Wang. 2018. Parameterized algorithms for edge biclique and related problems. *Theoretical Computer Science* 734 (2018), 105–118.
- [17] Fedor V Fomin and Petteri Kaski. 2013. Exact exponential algorithms. *Commun. ACM* 56, 3 (2013), 80–88.
- [18] Jian Gao, Zhenghang Xu, Ruizhi Li, and Minghao Yin. 2022. An exact algorithm with new upper bounds for the maximum  $k$ -defective clique problem in massive sparse graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 10174–10183.
- [19] Michael R Garey and David S Johnson. 1979. *Computers and intractability*. Vol. 174. Freeman San Francisco.
- [20] Nicolas Gillis and François Glineur. 2014. A continuous characterization of the maximum-edge biclique problem. *Journal of Global Optimization* 58, 3 (2014), 439–464.
- [21] Timo Gschwind, Stefan Irnich, and Isabel Podlinski. 2018. Maximum weight relaxed cliques and Russian Doll Search revisited. *Discrete Applied Mathematics* 234 (2018), 131–138.
- [22] Bryan Hooi, Kijung Shin, Hyun Ah Song, Alex Beutel, Neil Shah, and Christos Faloutsos. 2017. Graph-Based Fraud Detection in the Face of Camouflage. *ACM Trans. Knowl. Discov. Data* 11, 4 (2017), Article 44.
- [23] Bryan Hooi, Hyun Ah Song, Alex Beutel, Neil Shah, Kijung Shin, and Christos Faloutsos. 2016. Fraudar: Bounding graph fraud in the face of camouflage. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*. 895–904.
- [24] Mingming Jin, Jiongzhi Zheng, and Kun He. 2024. KD-Club: An Efficient Exact Algorithm with New Coloring-based Upper Bound for the Maximum  $k$ -Defective Clique Problem. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 20735–20742.
- [25] MA Langston, Elissa J Chesler, and Y Zhang. 2008. On finding bicliques in bipartite graphs: a novel algorithm with application to the integration of diverse biological data types. In *Proceedings of the 41st Annual Hawaii International Conference on System Sciences (HICSS 2008)(HICSS)*, Vol. 1. 473.
- [26] Sune Lehmann, Martin Schwartz, and Lars Kai Hansen. 2008. Biclique communities. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 78, 1 (2008), 016108.
- [27] Michael Ley. 2002. *The DBLP Computer Science Bibliography: Evolution, Research Issues, Perspectives (String Processing and Information Retrieval)*. Springer Berlin Heidelberg, 1–10.
- [28] Jingdong Li, Zhao Li, Xiaoling Wang, Xingjian Lu, Ji Zhang, and Hongyang Chen. 2023. GPU-Accelerated Maximal Bicliques Mining Framework for Large E-commerce Networks. , 539–544 pages.
- [29] Jinyan Li, Guimei Liu, Haiquan Li, and Limsoon Wong. 2007. Maximal biclique subgraphs and closed pattern pairs of the adjacency matrix: A one-to-one correspondence and mining algorithms. *IEEE Transactions on Knowledge and Data Engineering* 19, 12 (2007), 1625–1637.
- [30] Jinyan Li, Kelvin Sim, Guimei Liu, and Limsoon Wong. 2008. Maximal quasi-bicliques with balanced noise tolerance: Concepts and co-clustering applications.

In *Proceedings of the 2008 SIAM International Conference on Data Mining*. SIAM, 72–83.

- [31] Guimei Liu, Kelvin Sim, and Jinyan Li. 2006. Efficient mining of large maximal bicliques. In *Data Warehousing and Knowledge Discovery: 8th International Conference, DaWaK 2006, Krakow, Poland, September 4-8, 2006. Proceedings 8*. Springer, 437–448.
- [32] Chunyu Luo, Yi Zhou Zhengren Wang, and Mingyu Xiao. 2024. A Faster Branching Algorithm for the Maximum  $k$ -Defective Clique Problem. *arXiv preprint arXiv:2407.16588* (2024).
- [33] Bingqing Lyu, Lu Qin, Xuemin Lin, Ying Zhang, Zhengping Qian, and Jingren Zhou. 2020. Maximum biclique search at billion scale. *Proceedings of the VLDB Endowment* (2020).
- [34] Sara C Madeira and Arlindo L Oliveira. 2004. Biclustering algorithms for biological data analysis: a survey. *IEEE/ACM transactions on computational biology and bioinformatics* 1, 1 (2004), 24–45.
- [35] René Peeters. 2003. The maximum edge biclique problem is NP-complete. *Discrete Applied Mathematics* 131, 3 (2003), 651–654.
- [36] Amela Prelić, Stefan Bleuler, Philip Zimmermann, Anja Wille, Peter Bühlmann, Wilhelm Gruissem, Lars Hennig, Lothar Thiele, and Eckart Zitzler. 2006. A systematic comparison and evaluation of biclustering methods for gene expression data. *Bioinformatics* 22, 9 (2006), 1122–1129.
- [37] Eran Shoham, Honghai Yu, and Xiao-Li Li. 2016. On finding the maximum edge biclique in a bipartite graph: a subspace clustering approach. In *Proceedings of the 2016 SIAM International Conference on Data Mining*. SIAM, 315–323.
- [38] Shahrman Shahinpour, Shirin Shirvani, Zeynep Ertem, and Sergiy Butenko. 2017. Scale reduction techniques for computing maximum induced bicliques. *Algorithms* 10, 4 (2017), 113.
- [39] Vladimir Stozhkov, Austin Buchanan, Sergiy Butenko, and Vladimir Boginski. 2022. Continuous cubic formulations for cluster detection problems in networks. *Mathematical Programming* 196, 1 (2022), 279–307.
- [40] Melih Sözdinler and Can Özturan. 2018. Finding maximum edge biclique in bipartite networks by integer programming. In *2018 IEEE International Conference on Computational Science and Engineering (CSE)*. IEEE, 132–137.
- [41] Svyatoslav Trukhanov, Chitra Balasubramaniam, Balabhaskar Balasundaram, and Sergiy Butenko. 2013. Algorithms for detecting optimal hereditary structures in graphs, with application to clique relaxations. *Computational Optimization and Applications* 56 (2013), 113–130.
- [42] Kai Wang, Yiheng Hu, Xuemin Lin, Wenjie Zhang, Lu Qin, and Ying Zhang. 2021. A Cohesive Structure Based Bipartite Graph Analytics System. , 4799–4803 pages.
- [43] Kai Wang, Wenjie Zhang, Xuemin Lin, Lu Qin, and Alexander Zhou. 2022. Efficient personalized maximum biclique search. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 498–511.
- [44] Yiyuan Wang, Shaowei Cai, and Minghao Yin. 2018. New heuristic approaches for maximum balanced biclique problem. *Information Sciences* 432 (2018), 362–375.
- [45] Haiyuan Yu, Alberto Paccanaro, Valery Trifonov, and Mark Gerstein. 2006. Predicting interactions in protein networks by completing defective cliques. *Bioinformatics* 22, 7 (2006), 823–829.
- [46] Kaiqiang Yu and Cheng Long. 2023. Maximum k-Biplex Search on Bipartite Graphs: A Symmetric-BK Branching Approach. *Proc. ACM Manag. Data* 1, 1 (2023), Article 49.
- [47] Bo Yuan, Bin Li, Huanhuan Chen, and Xin Yao. 2015. A New Evolutionary Algorithm with Structure Mutation for the Maximum Balanced Biclique Problem. *IEEE Transactions on Cybernetics* 45, 5 (2015), 1054–1067.
- [48] Yi Zhou and Jin-Kao Hao. 2019. Tabu search with graph reduction for finding maximum balanced bicliques in bipartite graphs. *Eng. Appl. Artif. Intell.* 77 (2019), 86–97.